FRE7241 Algorithmic Portfolio Management

Lecture#6, Fall 2025

Jerzy Pawlowski *jp3900@nyu.edu*

NYU Tandon School of Engineering

December 2, 2025

NYU | TANDON SCHOOL OF ENGINEERING

The Covariance of Stock Returns

Estimating the covariance of stock returns is complicated because their date ranges may not overlap in time. Stocks may trade over different date ranges because of IPOs and corporate events (takeovers, mergers).

The function `cov()` calculates the covariance matrix of time series. The argument `use="pairwise.complete.obs"` removes `NA` values from pairs of stock returns.

But removing `NA` values in pairs of stock returns can produce covariance matrices which are not positive semi-definite.

The reason is because the covariance are calculated over different time intervals for different pairs of stock returns.

Matrices which are not positive semi-definite may not have an inverse matrix, but they have a generalized inverse.

The function `MASS::ginv()` calculates the generalized inverse of a matrix.

```
> # Select all the ETF symbols except "VXX", "SVXY" "MTUM", "QUAL",
> symbolv <- colnames(rutils::etfenv$returns)
> symbolv <- symbolv[!(symbolv %in% c("VXX", "SVXY", "MTUM", "QUAL"
> # Extract columns of rutils::etfenv$returns and overwrite NA value
> retp <- rutils::etfenv$returns["1999/", symbolv]
> sum(is.na(retp)) ##  Number of NA values
> nstocks <- NCOL(retp)
> datev <- zoo::index(retp)
> # Calculate the covariance ignoring NA values
> covmat <- cov(retp, use="pairwise.complete.obs")
> sum(is.na(covmat))
> # Calculate the inverse of covmat
> invmat <- solve(covmat)
> round(invmat %*% covmat, digits=5)
> # Calculate the generalized inverse of covmat
> invreg <- MASS::ginv(covmat)
> all.equal(unname(invmat), invreg)
```

Generalized Inverse of Singular Covariance Matrices

The standard inverse of a positive semi-definite matrix $\mathbb{C}$ can be calculated from its *eigenvalues* $\mathbb{D}$ and its *eigenvectors* $\mathbb{O}$ as follows:

$$\mathbb{C}^{-1} = \mathbb{O}\,\mathbb{D}^{-1}\,\mathbb{O}^{T}$$

The covariance matrix may not be positive semi-definite if the number of time periods of returns (rows) is less than the number of stocks (columns).

In that case some of the higher order eigenvalues are zero, and the above covariance matrix inverse is singular.

But a non-positive semi-definite covariance matrix may still have a *generalized inverse*.

The *generalized inverse* $\mathbb{C}_g^{-1}$ is calculated by removing the zero eigenvalues, and keeping only the first n non-zero *eigenvalues*:

$$\mathbb{C}_g^{-1} = \mathbb{O}_n\,\mathbb{D}_n^{-1}\,\mathbb{O}_n^{T}$$

Where $\mathbb{D}_n$ and $\mathbb{O}_n$ are matrices with the higher order eigenvalues and eigenvectors removed.

The generalized inverse $\mathbb{C}_g^{-1}$ of the matrix $\mathbb{C}$ satisfies the equation:

$$\mathbb{C}\,\mathbb{C}_g^{-1}\mathbb{C} = \mathbb{C}$$

Which is a generalization of the standard inverse property: $\mathbb{C}^{-1}\mathbb{C} = \mathbb{1}$

```
> # Create rectangular matrix with collinear columns
> matv <- matrix(rnorm(10*8), nc=10)
> # Calculate covariance matrix
> covmat <- cov(matv)
> # Calculate inverse of covmat - error
> invmat <- solve(covmat)
> # Perform eigen decomposition
> eigend <- eigen(covmat)
> eigenvec <- eigend$vectors
> eigenval <- eigend$values
> # Set tolerance for determining zero singular values
> precv <- sqrt(.Machine$double.eps)
> # Calculate generalized inverse from the eigen decomposition
> notzero <- (eigenval > (precv*eigenval[1]))
> inveigen <- eigenvec[, notzero] %*%
+   (t(eigenvec[, notzero]) / eigenval[notzero])
> # Inverse property of inveigen isn't satisfied
> all.equal(inveigen %*% covmat, diag(NROW(covmat)))
> # Generalized inverse property of inveigen is satisfied
> all.equal(covmat %*% inveigen %*% covmat, covmat)
> # Calculate generalized inverse using MASS::ginv()
> invreg <- MASS::ginv(covmat)
> # Verify that inveigen is the same as invreg
> all.equal(inveigen, invreg)
```

Portfolio Optimization Strategy

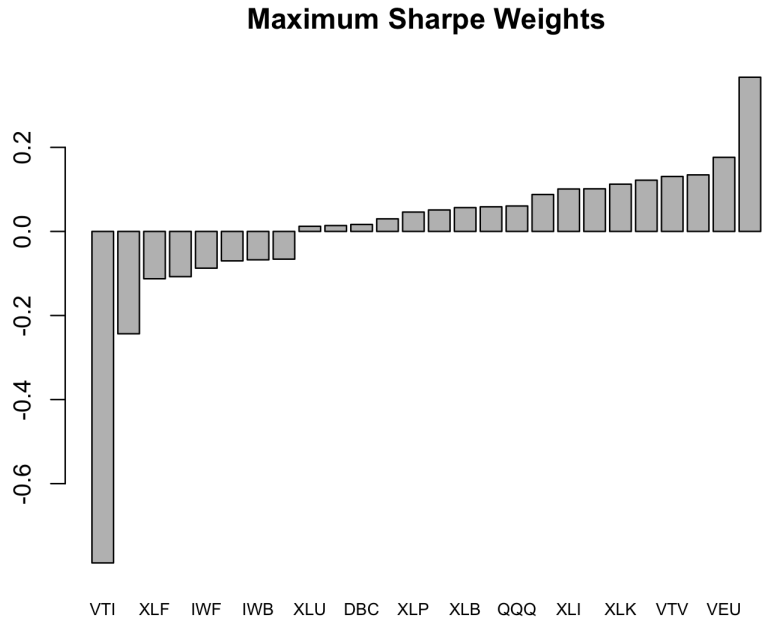
The optimal portfolio weights $\mathbf{w}$ are equal to the past in-sample excess returns $\mu = \mathbf{r} - r_f$ (in excess of the risk-free rate r_f) multiplied by the inverse of the covariance matrix $\mathbb{C}$:

$$\mathbf{w} = \mathbb{C}^{-1}\mu$$

The *portfolio optimization* strategy invests in the best performing portfolio in the past *in-sample* interval, expecting that it will continue performing well *out-of-sample*.

The *portfolio optimization* strategy consists of:

1. Calculating the maximum Sharpe ratio portfolio weights in the *in-sample* interval,
2. Applying the weights and calculating the portfolio returns in the *out-of-sample* interval.




```
> # Maximum Sharpe weights in-sample interval
> retis <- retp["/2014"]
> raterf <- 0.03/252
> retx <- (retis - raterf)
> invreg <- MASS::ginv(cov(retis, use="pairwise.complete.obs"))
> weightv <- drop(invreg %*% colMeans(retx, na.rm=TRUE))
> weightv <- weightv/sqrt(sum(weightv^2))
> names(weightv) <- colnames(retp)
```

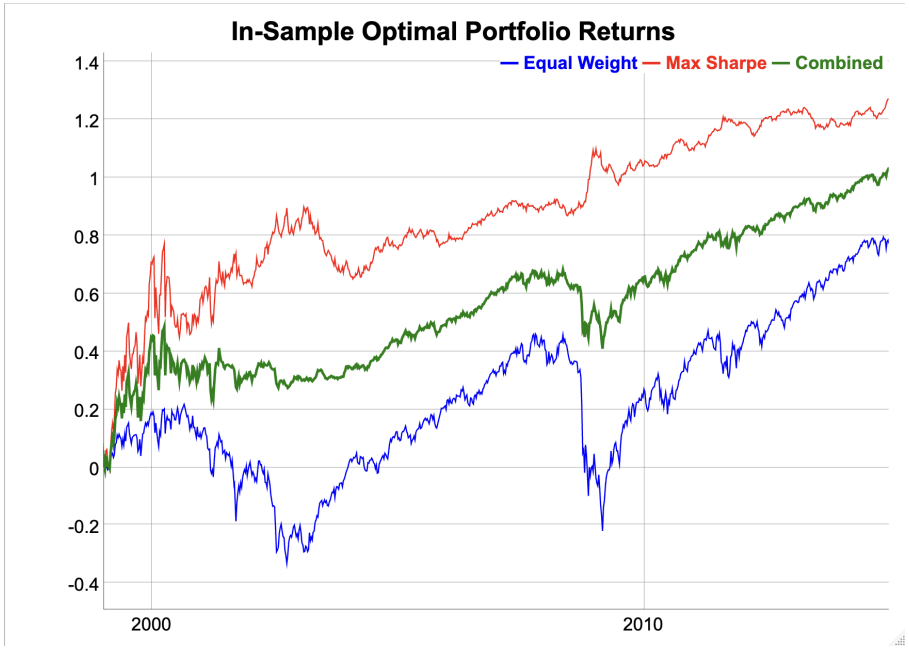
```
> # Plot portfolio weights
> barplot(sort(weightv), main="Maximum Sharpe Weights", cex.names=0
```

Portfolio Optimization Strategy In-Sample

The in-sample performance of the *maximum Sharpe* portfolio is much better than the equal weight portfolio.

The function `HighFreq::mult_mat()` multiplies element-wise the rows or columns of a matrix times a vector.

```
> # Calculate the equal weight index
> retew <- xts::xts(rowMeans(retp, na.rm=TRUE), datev)
> # Calculate the in-sample weighted returns using Rcpp
> pnlis <- HighFreq::mult_mat(weightv, retis)
> # Or using the transpose
> # pnlis <- unname(t(t(retis)*weightv))
> pnlis <- rowMeans(pnlis, na.rm=TRUE)
> pnlis <- pnlis*sd(retew["/2014"])/sd(pnlis)
```



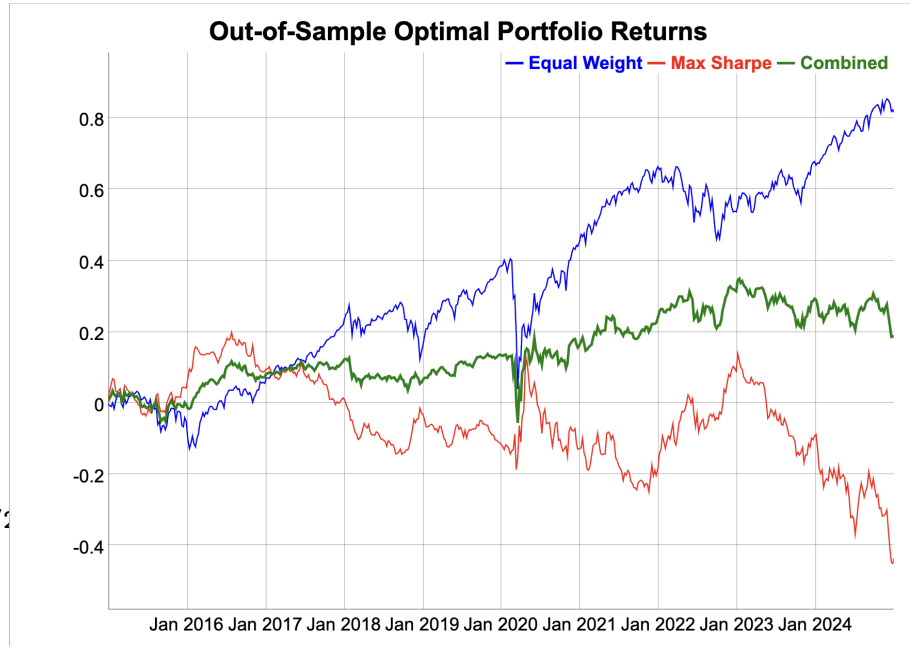

```
> # Calculate the Sharpe and Sortino ratios
> wealthv <- cbind(retew["/2014"], pnlis, (pnlis + retew["/2014"])/2
> colnames(wealthv) <- c("EqualWeight", "Max Sharpe", "Combined")
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Dygraph cumulative wealth
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="In-Sample Optimal Portfolio Returns") %>%
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dySeries(name="Combined", strokeWidth=2) %>%
+   dyLegend(width=300)
```

Portfolio Optimization Strategy Out-of-Sample

The out-of-sample performance of the optimal in-sample portfolio is not nearly as good as in-sample.

Combining the *maximum Sharpe* portfolio with the equal weight portfolio produces and even better performing portfolio.

```
> # Calculate the equal weight index
> retos <- retp["2015/"]
> # Calculate out-of-sample portfolio returns
> pnlos <- HighFreq::mult_mat(weightv, retos)
> pnlos <- rowMeans(pnlos, na.rm=TRUE)
> pnlos <- pnlos*sd(retew["2015/"])/sd(pnlos)
> wealthv <- cbind(retew["2015/"], pnlos, (pnlos + retew["2015/"])/2
> colnames(wealthv) <- c("EqualWeight", "Max Sharpe", "Combined")
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
```



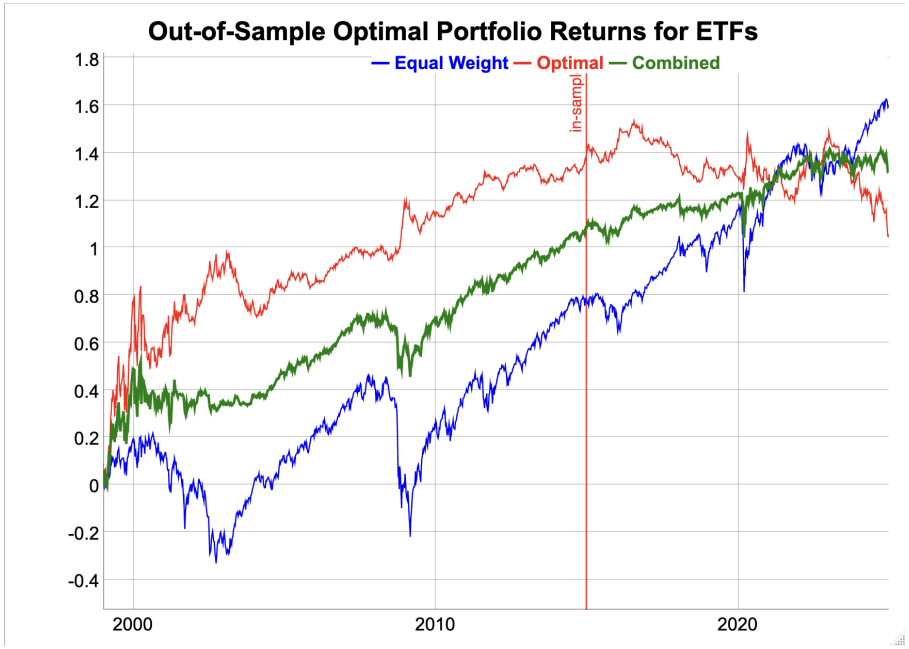

```
> # Dygraph cumulative wealth
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Out-of-Sample Optimal Portfolio Returns") %>%
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dySeries(name="Combined", strokeWidth=2) %>%
+   dyLegend(width=300)
```

Portfolio Optimization Strategy for ETFs

The *portfolio optimization* strategy for ETFs is *overfit* in the *in-sample* interval.

Therefore the strategy doesn't perform as well in the *out-of-sample* interval as in the *in-sample* interval.

```
> # Maximum Sharpe weights in-sample interval
> invreg <- MASS::ginv(cov(retis, use="pairwise.complete.obs"))
> weightv <- invreg %*% colMeans(retx, na.rm=TRUE)
> names(weightv) <- colnames(retp)
> # Calculate cumulative wealth
> pnls <- HighFreq::mult_mat(weightv, retp)
> pnls <- rowMeans(pnls, na.rm=TRUE)
> pnls <- pnls*sd(retew)/sd(pnls)
> wealthv <- cbind(retew, pnls, (pnls + retew)/2)
> colnames(wealthv) <- c("EqualWeight", "Optimal", "Combined")
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
```




```
> # Dygraph cumulative wealth
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Out-of-Sample Optimal Portfolio Returns for ETFs") %>%
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dySeries(name="Combined", strokeWidth=2) %>%
+   dyEvent(zoo::index(last(retis[, 1])), label="in-sample", strokeP
+   dyLegend(width=400)
```

Dimension Reduction of the Covariance Matrix

If the higher order singular values are very small then the inverse matrix amplifies the statistical noise in the response matrix.

The *reduced inverse* $\mathbb{C}_R^{-1}$ is calculated from the largest (lowest order) eigenvalues, up to $dimax = d$:

$$\mathbb{C}_R^{-1} = \mathbb{O}_d \, \mathbb{D}_d^{-1} \, \mathbb{O}_d^T$$

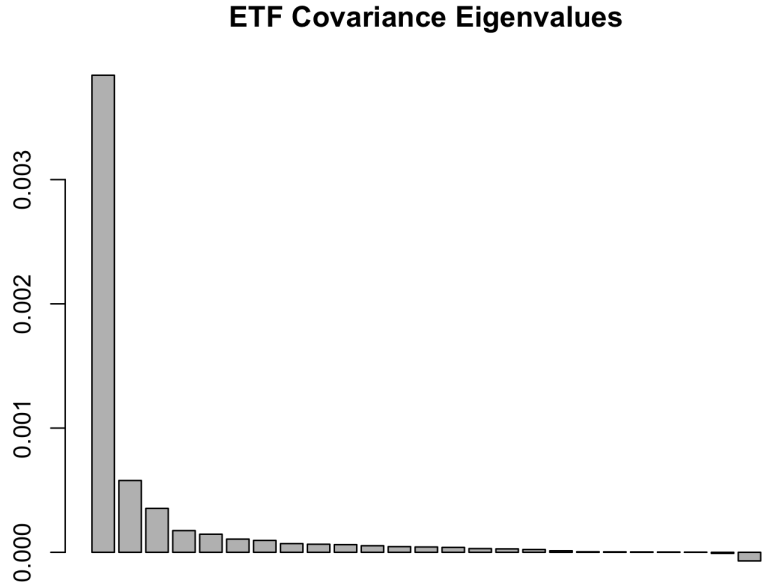
The parameter *dimax* specifies the number of eigenvalues used for calculating the *reduced inverse* of the covariance matrix of returns.

The *dimension reduction* technique calculates the *reduced inverse* of a covariance matrix by removing the very small, higher order eigenvalues, to reduce the propagation of statistical noise and improve the signal-to-noise ratio:

Even though the *reduced inverse* $\mathbb{C}_R^{-1}$ does not satisfy the matrix inverse property (so it's biased), its out-of-sample forecasts are usually more accurate than those using the exact inverse matrix.

But removing a larger number of eigenvalues increases the bias of the covariance matrix, which is an example of the *bias-variance tradeoff*.

The optimal value of the parameter *dimax* can be determined using *backtesting* (*cross-validation*).



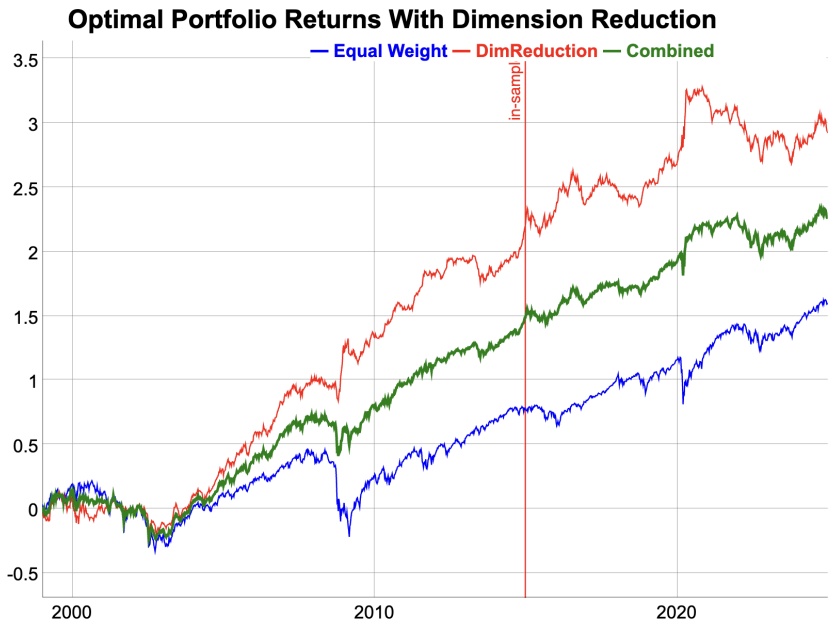

```
> # Calculate in-sample covariance matrix
> covmat <- cov(retis, use="pairwise.complete.obs")
> eigend <- eigen(covmat)
> eigenvec <- eigend$vectors
> eigenval <- eigend$values
> # Plot the eigenvalues
> barplot(eigenval, main="ETF Covariance Eigenvalues", cex.names=0.7
> # Calculate reduced inverse of covariance matrix
> dimax <- 9
> invred <- eigenvec[, 1:dimax] %*%
+   (t(eigenvec[, 1:dimax]) / eigenval[1:dimax])
> # Reduced inverse does not satisfy matrix inverse property
> all.equal(covmat %*% invred %*% covmat, covmat)
```

Portfolio Optimization for ETFs with Dimension Reduction

The *out-of-sample* performance of the *portfolio optimization* strategy can be improved by applying dimension reduction to the inverse of the covariance matrix.

The *in-sample* performance is worse because dimension reduction reduces *overfitting*.

```
> # Calculate portfolio weights
> weightv <- drop(invred %*% colMeans(retis, na.rm=TRUE))
> names(weightv) <- colnames(retp)
> # Calculate cumulative wealth
> pnls <- HighFreq::mult_mat(weightv, retp)
> pnls <- rowMeans(pnls, na.rm=TRUE)
> pnls <- pnls*sd(retew)/sd(pnls)
> wealthv <- cbind(retew, pnls, (pnls + retew)/2)
> colnames(wealthv) <- c("EqualWeight", "DimReduction", "Combined")
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
```




```
> # Dygraph cumulative wealth
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Optimal Portfolio Returns With Dimension Reduction") %>%
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dySeries(name="Combined", strokeWidth=2) %>%
+   dyEvent(zoo::index(last(retis[, 1])), label="in-sample", strokeP
+   dyLegend(width=400)
```

Portfolio Optimization With Return Shrinkage

To further reduce the statistical noise, the individual returns r_i can be *shrunk* to the average portfolio returns $\bar{r}$:

$$r_i' = (1 - \alpha)\, r_i + \alpha\, \bar{r}$$

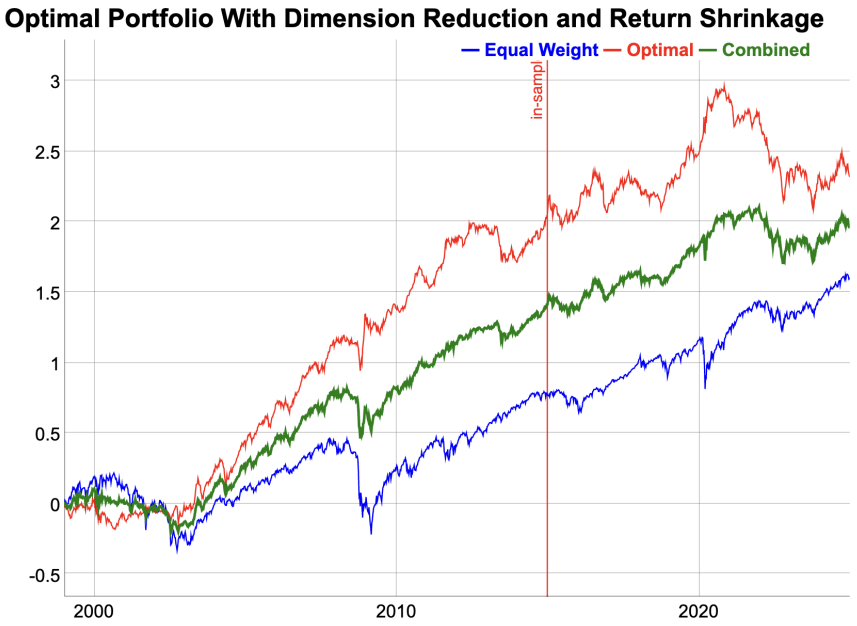
The parameter α is the *shrinkage* intensity, and it determines the strength of the *shrinkage* of individual returns to their mean.

If $\alpha = 0$ then there is no *shrinkage*, while if $\alpha = 1$ then all the returns are *shrunk* to their common mean: $r_i = \bar{r}$.

If $\alpha = 1$ then the weights are equal to the minimum variance portfolio weights. So an intermediate value of α gives weights that are an average of the maximum Sharpe weights and the minimum variance weights.

The optimal value of the *shrinkage* intensity α can be determined using *backtesting* (*cross-validation*).

```
> # Shrink the in-sample returns to their mean
> alphac <- 0.7
> retxm <- rowMeans(retx, na.rm=TRUE)
> retxis <- (1-alphac)*retx + alphac*retxm
> # Calculate portfolio weights
> weightv <- invred %*% colMeans(retxis, na.rm=TRUE)
> # Calculate cumulative wealth
> pnls <- HighFreq::mult_mat(weightv, retp)
> pnls <- rowMeans(pnls, na.rm=TRUE)
> pnls <- pnls*sd(retew)/sd(pnls)
> wealthv <- cbind(retew, pnls, (pnls + retew)/2)
> colnames(wealthv) <- c("EqualWeight", "Optimal", "Combined")
```




```
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Dygraph cumulative wealth
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Optimal Portfolio With Dimension Reduction and Return Shri
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dySeries(name="Combined", strokeWidth=2) %>%
+   dyEvent(zoo::index(last(retis[, 1])), label="in-sample", strokeP
+   dyLegend(width=300)
```

Rolling Portfolio Optimization Strategy

In a *rolling portfolio optimization strategy*, the portfolio is optimized periodically and held out-of-sample.

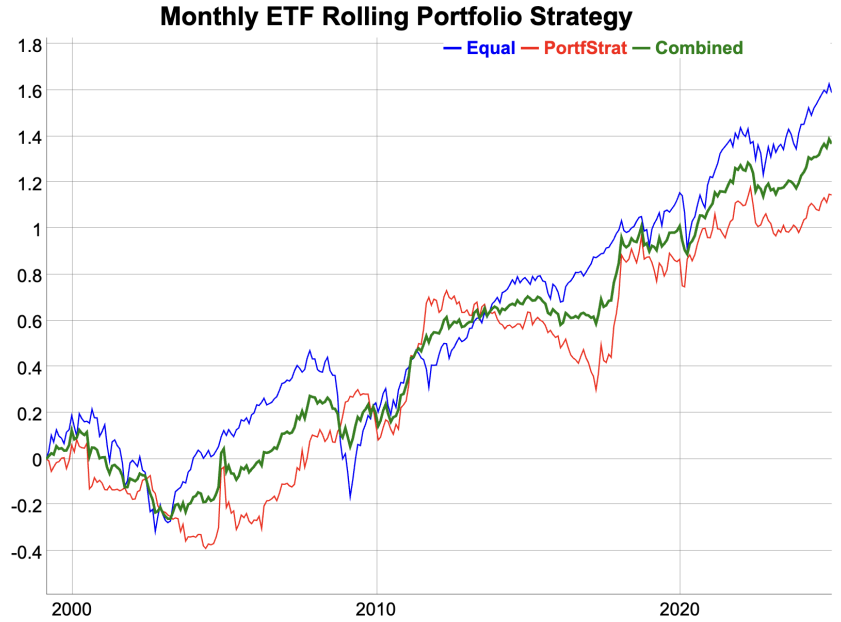
- Calculate the *end points* for portfolio rebalancing,
- Define an objective function for optimizing the portfolio weights,
- Calculate the optimal portfolio weights from the past (in-sample) performance,
- Calculate the out-of-sample returns by applying the portfolio weights to the future returns.

```
> # Define monthly end and start points
> retx <- (retp - raterf)
> endd <- rutils::calc_endpoints(retp, interval="months")
> endd <- endd[endd > (nstocks+1)]
> npts <- NROW(endd)
> lookb <- 3
> startp <- c(rep_len(0, lookb), endd[1:(npts-lookb)])
> # Perform loop over end points
> pnls <- lapply(1:(npts-1), function(tday) {
+     ##  Calculate the portfolio weights
+     retis <- retx[startp[tday]:endd[tday], ]
+     covmat <- cov(retis, use="pairwise.complete.obs")
+     covmat[is.na(covmat)] <- 0
+     invreg <- MASS::ginv(covmat)
+     colm <- colMeans(retis, na.rm=TRUE)
+     colm[is.na(colm)] <- 0
+     weightv <- invreg %*% colm
+     ##  Calculate the in-sample portfolio returns
+     pnlis <- HighFreq::mult_mat(weightv, retis)
+     pnlis <- rowMeans(pnlis, na.rm=TRUE)
+     ##  Calculate the out-of-sample portfolio returns
+     retos <- retp[(endd[tday]+1):endd[tday+1], ]
+     pnlos <- HighFreq::mult_mat(weightv, retos)
+     pnlos <- rowMeans(pnlos, na.rm=TRUE)
+     xts::xts(pnlos, zoo::index(retos))
+ })  ##  end lapply
> pnls <- do.call(rbind, pnls)
> pnls <- rbind(retew[paste0("/", start(pnls)-1)], pnls)
```

Rolling Portfolio Strategy Performance

In a *rolling portfolio optimization strategy*, the portfolio is optimized periodically and held out-of-sample.

```
> # Calculate the Sharpe and Sortino ratios
> pnls <- pnls*sd(retew)/sd(pnls)
> wealthv <- cbind(retew, pnls, (pnls + retew)/2)
> colnames(wealthv) <- c("EqualWeight", "PortfStrat", "Combined")
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
```

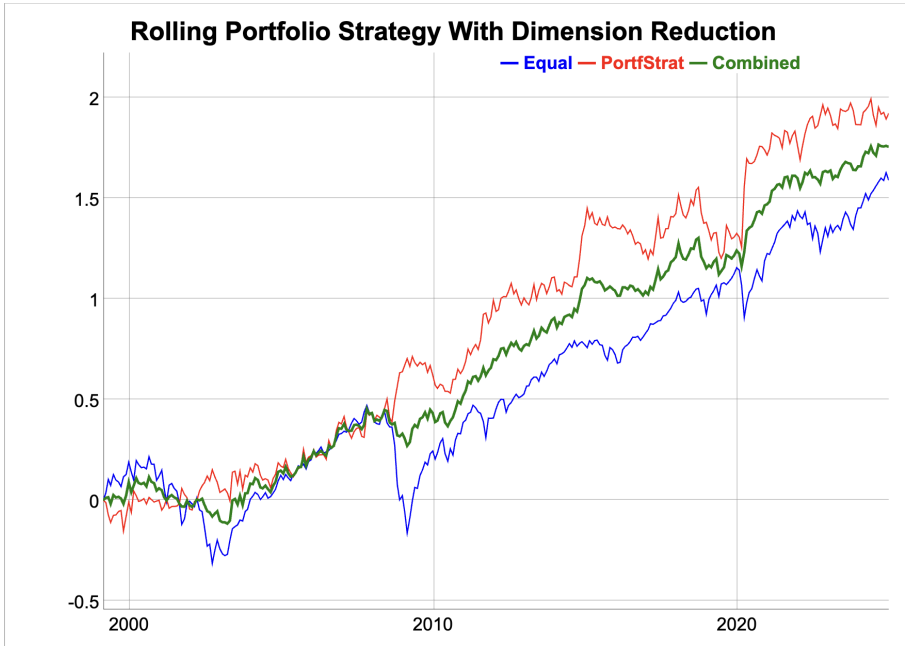



```
> # Dygraph cumulative wealth
> dygraphs::dygraph(cumsum(wealthv)[endw], main="Monthly ETF Rolling
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dySeries(name="Combined", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Rolling Portfolio Strategy With Dimension Reduction

Dimension reduction improves the performance of the rolling portfolio strategy because it suppresses the data noise.

The strategy performed especially well during sharp market selloffs, like in the years 2008 and 2020.




```
> # Perform loop over end points
> dimax <- 9
> pnls <- lapply(1:(npts-1), function(tday) {
+     ##  Calculate the portfolio weights
+     retis <- retx[startp[tday]:endd[tday], ]
+     covmat <- cov(retis, use="pairwise.complete.obs")
+     covmat[is.na(covmat)] <- 0
+     eigend <- eigen(covmat)
+     eigenvec <- eigend$vectors
+     eigenval <- eigend$values
+     invred <- eigenvec[, 1:dimax] %*%
+ (t(eigenvec[, 1:dimax]) / eigenval[1:dimax])
+     colm <- colMeans(retis, na.rm=TRUE)
+     colm[is.na(colm)] <- 0
+     weightv <- invred %*% colm
+     ##  Calculate the in-sample portfolio returns
+     pnlis <- HighFreq::mult_mat(weightv, retis)
+     pnlis <- rowMeans(pnlis, na.rm=TRUE)
+     weightv <- weightv*0.01/sd(pnlis)
+     ##  Calculate the out-of-sample portfolio returns
+     retos <- retp[(endd[tday]+1):endd[tday+1], ]
+     pnlos <- HighFreq::mult_mat(weightv, retos)
+     pnlos <- rowMeans(pnlos, na.rm=TRUE)
+     xts::xts(pnlos, zoo::index(retos))
+ })  ##  end lapply
> pnls <- do.call(rbind, pnls)
> pnls <- rbind(retew[paste0("/", start(pnls)-1)], pnls)
```

```
> # Calculate the Sharpe and Sortino ratios
> pnls <- pnls*sd(retew)/sd(pnls)
> wealthv <- cbind(retew, pnls, (pnls + retew)/2)
> colnames(wealthv) <- c("EqualWeight", "PortfStrat", "Combined")
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Dygraph cumulative wealth
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Rolling Portfolio Strategy With Dimension Reduction") %>%
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dySeries(name="Combined", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Rolling Portfolio Strategy With Return Shrinkage

Shrinkage averages the stock returns, which creates bias, but it also reduces the variance.

Return shrinkage can be applied to improve the performance of the rolling portfolio strategy.

```
> alphac <- 0.7 ##  Return shrinkage intensity
> # Perform loop over end points
> pnls <- lapply(1:(npts-1), function(tday) {
+     ##  Shrink the in-sample returns to their mean
+     retis <- retx[startp[tday]:endd[tday], ]
+     rowm <- rowMeans(retis, na.rm=TRUE)
+     rowm[is.na(rowm)] <- 0
+     retis <- (1-alphac)*retis + alphac*rowm
+     ##  Calculate the portfolio weights
+     covmat <- cov(retis, use="pairwise.complete.obs")
+     covmat[is.na(covmat)] <- 0
+     eigend <- eigen(covmat)
+     eigenvec <- eigend$vectors
+     eigenval <- eigend$values
+     invred <- eigenvec[, 1:dimax] %*%
+ (t(eigenvec[, 1:dimax]) / eigenval[1:dimax])
+     colm <- colMeans(retis, na.rm=TRUE)
+     colm[is.na(colm)] <- 0
+     weightv <- invred %*% colm
+     ##  Scale the weights to volatility target
+     pnlis <- HighFreq::mult_mat(weightv, retis)
+     pnlis <- rowMeans(pnlis, na.rm=TRUE)
+     weightv <- weightv*0.01/sd(pnlis)
+     ##  Calculate the out-of-sample portfolio returns
+     retos <- retp[(endd[tday]+1):endd[tday+1], ]
+     pnlos <- HighFreq::mult_mat(weightv, retos)
+     pnlos <- rowMeans(pnlos, na.rm=TRUE)
+     xts::xts(pnlos, zoo::index(retos))
+ })  ##  end lapply
> pnls <- do.call(rbind, pnls)
> pnls <- rbind(retew[paste0("/", start(pnls)-1)], pnls)
```


```
> # Calculate the Sharpe and Sortino ratios
> pnls <- pnls*sd(retew)/sd(pnls)
> wealthv <- cbind(retew, pnls, (pnls + retew)/2)
> colnames(wealthv) <- c("EqualWeight", "PortfStrat", "Combined")
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Dygraph cumulative wealth
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Rolling Portfolio Strategy With Return Shrinkage") %>%
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dySeries(name="Combined", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

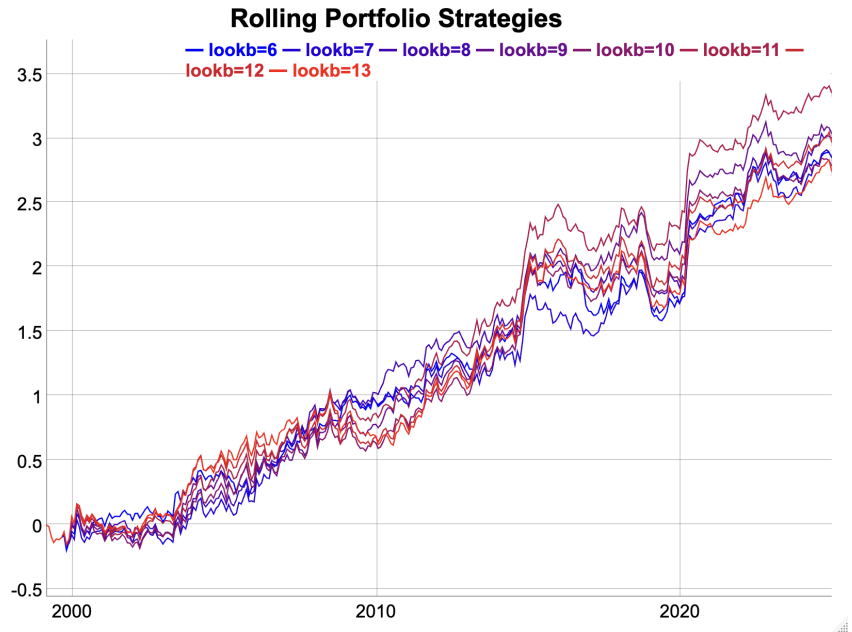
Function for Rolling Portfolio Optimization Strategy

```
> # Define backtest functional for rolling portfolio strategy
> roll_portf <- function(retx, ##  Excess returns
+                  retp, ##  Stock returns
+                  endd, ##  End points
+                  lookb=12, ##  Look-back interval
+                  dimax=3, ##  Dimension reduction parameter
+                  alphac=0.0, ##  Return shrinkage intensity
+                  bidask=0.0, ##  Bid-offer spread
+                  ...) {
+   npts <- NROW(endd)
+   startp <- c(rep_len(0, lookb), endd[1:(npts-lookb)])
+   pnls <- lapply(1:(npts-1), function(tday) {
+     retis <- retx[startp[tday]:endd[tday], ]
+     ##  Shrink the in-sample returns to their mean
+     if (alphac > 0) {
+ rowm <- rowMeans(retis, na.rm=TRUE)
+ rowm[is.na(rowm)] <- 0
+ retis <- (1-alphac)*retis + alphac*rowm
+     } ##  end if
+     ##  Calculate the portfolio weights
+     covmat <- cov(retis, use="pairwise.complete.obs")
+     covmat[is.na(covmat)] <- 0
+     eigend <- eigen(covmat)
+     eigenvec <- eigend$vectors
+     eigenval <- eigend$values
+     invred <- eigenvec[, 1:dimax] %*% (t(eigenvec[, 1:dimax]) / eigenval[1:dimax])
+     colm <- colMeans(retis, na.rm=TRUE)
+     colm[is.na(colm)] <- 0
+     weightv <- invred %*% colm
+     ##  Scale the weights to volatility target
+     pnlis <- HighFreq::mult_mat(weightv, retis)
+     pnlis <- rowMeans(pnlis, na.rm=TRUE)
+     weightv <- weightv*0.01/sd(pnlis)
+     ##  Calculate the out-of-sample portfolio returns
+     retos <- retp[(endd[tday]+1):endd[tday+1], ]
+     pnlos <- HighFreq::mult_mat(weightv, retos)
+     pnlos <- rowMeans(pnlos, na.rm=TRUE)
+     xts::xts(pnlos, zoo::index(retos))
```

Rolling Portfolio Optimization With Different Look-backs

Multiple *rolling portfolio optimization* strategies can be backtested by calling the function `roll_portf()` in a loop over a vector of *look-back* parameters.

```
> # Simulate a monthly ETF portfolio strategy
> pnls <- roll_portf(retx=retx, retp=retp, endd=endd,
+   lookb=lookb, dimax=dimax)
> # Perform sapply loop over lookbv
> lookbv <- seq(6, 13, by=1)
> library(parallel)  ##  Load package parallel
> ncores <- detectCores() - 1
> pnls <- mclapply(lookbv, roll_portf, retp=retp, retx=retx,
+   endd=endd, dimax=dimax, mc.cores=ncores)
> pnls <- do.call(cbind, pnls)
> colnames(pnls) <- paste0("lookb=", lookbv)
> profilev <- sapply(pnls, sum)
> lookb <- lookbv[which.max(profilev)]
```

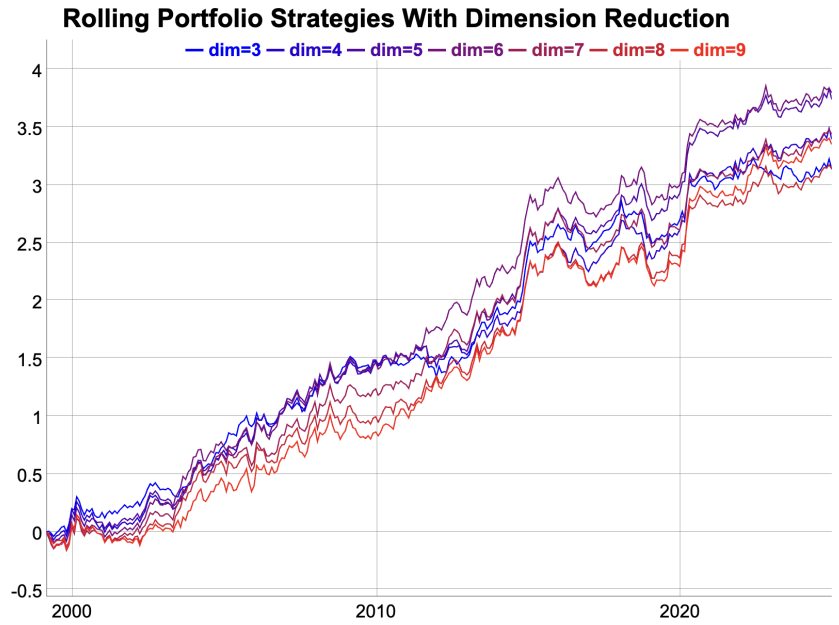



```
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(pnls, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Plot dygraph of monthly ETF portfolio strategies
> colorv <- colorRampPalette(c("blue", "red"))(NCOL(pnls))
> dygraphs::dygraph(cumsum(pnls)[endw], main="Rolling Portfolio Stra
+   dyOptions(colors=colorv, strokeWidth=1) %>%
+   dyLegend(show="always", width=500)
> # Plot portfolio strategies using quantmod
> themev <- chart_theme()
> themev$col$line.col <-
+   colorRampPalette(c("blue", "red"))(NCOL(pnls))
> quantmod::chart_Series(cumsum(pnls),
+   theme=themev, name="Rolling Portfolio Strategies")
> legend("bottomleft", legend=colnames(pnls),
+   inset=0.02, bg="white", cex=0.7, lwd=rep(6, NCOL(retp)),
+   col=themev$col$line.col, bty="n")
```

Rolling Portfolio Optimization With Different Dimension Reduction

Multiple *rolling portfolio optimization* strategies can be backtested by calling the function `roll_portf()` in a loop over a vector of the dimension reduction parameter.

```
> # Perform backtest for different dimax values
> dimv <- 3:9
> pnls <- mclapply(dimv, roll_portf, retp=retp, retx=retx,
+   endd=endd, lookb=lookb, mc.cores=ncores)
> pnls <- do.call(cbind, pnls)
> colnames(pnls) <- paste0("dim=", dimv)
> profilev <- sapply(pnls, sum)
> dimax <- dimv[which.max(profilev)]
```



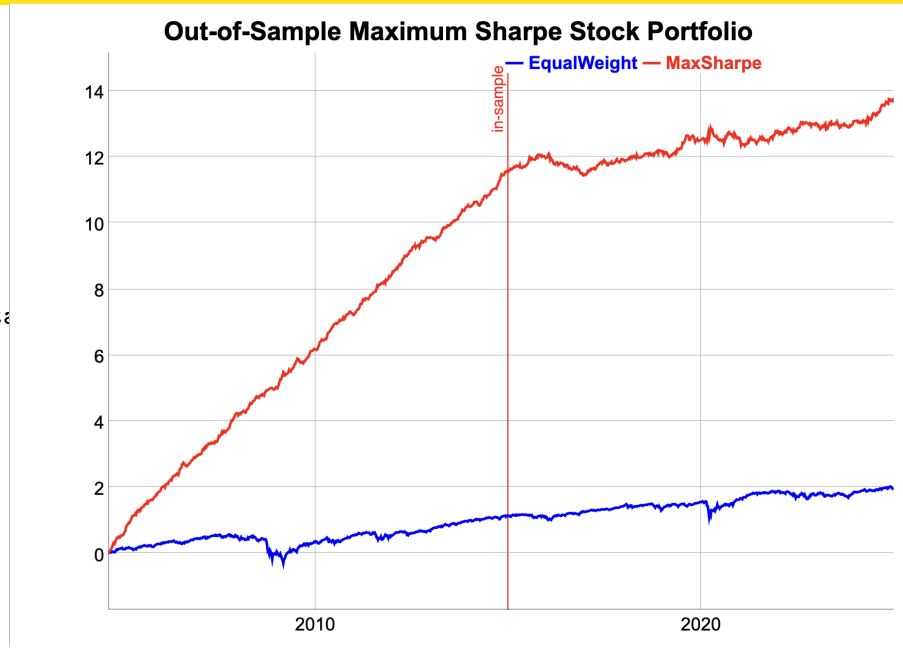

```
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(pnls, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Plot dygraph of monthly ETF portfolio strategies
> colorv <- colorRampPalette(c("blue", "red"))(NCOL(pnls))
> dygraphs::dygraph(cumsum(pnls)[endw],
+   main="Rolling Portfolio Strategies With Dimension Reduction") %>
+   dyOptions(colors=colorv, strokeWidth=1) %>%
+   dyLegend(show="always", width=500)
> # Plot portfolio strategies using quantmod
> themev <- chart_theme()
> themev$col$line.col <-
+   colorRampPalette(c("blue", "red"))(NCOL(pnls))
> quantmod::chart_Series(cumsum(pnls),
+   theme=themev, name="Rolling Portfolio Strategies")
> legend("bottomleft", legend=colnames(pnls),
+   inset=0.02, bg="white", cex=0.7, lwd=rep(6, NCOL(retp)),
+   col=themev$col$line.col, bty="n")
```

Optimal Stock Portfolio Out-of-Sample

The *portfolio optimization* strategy for stocks is *overfit* in the *in-sample* interval.

Therefore the strategy performance is poor in the *out-of-sample* interval.

```
> # Load stock returns
> load("/Users/jerzy/Develop/lecture_slides/data/sp500_returns.RDat
> datev <- zoo::index(na.omit(retstock$GOOGL))
> retp <- retstock[datev] ##  Subset the returns to GOOGL
> # Remove the stocks with any NA values
> numna <- sapply(retp, function(x) sum(is.na(x)))
> retp <- retp[, numna==0]
> nstocks <- NCOL(retp)
> datev <- zoo::index(retp)
> symbolv <- colnames(retp)
> retis <- retp["/2014"] ##  In-sample returns
> raterf <- 0.03/252
> retx <- (retis - raterf) ##  Excess returns
> # Maximum Sharpe weights in-sample interval
> colmeanv <- colMeans(retx, na.rm=TRUE)
> covmat <- cov(retis, use="pairwise.complete.obs")
> invreg <- MASS::ginv(covmat)
> weightv <- drop(invreg %*% colmeanv)
> names(weightv) <- symbolv
> head(sort(weightv))
> tail(sort(weightv))
> # Calculate the portfolio returns
> pnls <- HighFreq::mult_mat(weightv, retp)
> pnls <- rowMeans(pnls, na.rm=TRUE)
> pnls <- xts::xts(pnls, datev)
> retew <- xts::xts(rowMeans(retp, na.rm=TRUE), datev)
> pnls <- pnls*sd(retew["/2014"])/sd(pnls["/2014"])
> wealthv <- cbind(retew, pnls)
> colnames(wealthv) <- c("EqualWeight", "MaxSharpe")
```



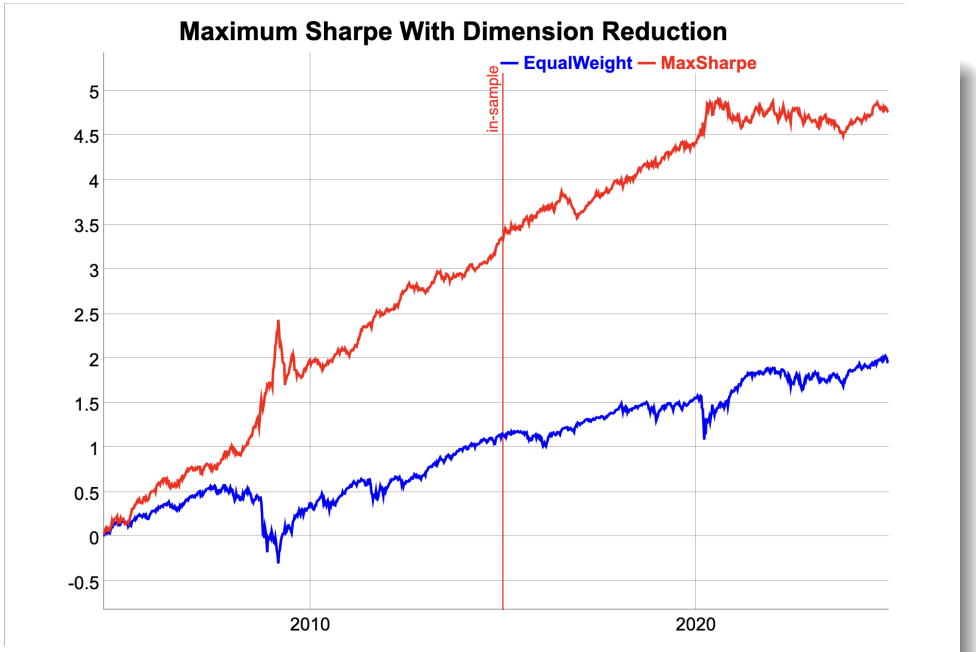

```
> # Calculate the in-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv["/2014"], function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Calculate the out-of-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv["2015/"], function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Plot of cumulative portfolio returns
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Out-of-Sample Maximum Sharpe Stock Portfolio") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyEvent(zoo::index(last(retis[, 1])), label="in-sample", strokeP
+   dyLegend(width=300)
```

Maximum Sharpe Portfolio With Dimension Reduction

The *out-of-sample* performance of the *portfolio optimization* strategy can be improved by applying dimension reduction to the inverse of the covariance matrix.

The *in-sample* performance is worse because dimension reduction reduces *overfitting*.

```
> # Calculate reduced inverse of covariance matrix
> dimax <- 10
> eigend <- eigen(covmat)
> eigenvec <- eigend$vectors
> eigenval <- eigend$values
> invred <- eigenvec[, 1:dimax] %*%
+   (t(eigenvec[, 1:dimax]) / eigenval[1:dimax])
> # Calculate portfolio weights and PnLs
> colmeanv <- colMeans(retx["/2014"], na.rm=TRUE)
> weightv <- invred %*% colmeanv
> pnls <- HighFreq::mult_mat(weightv, retp)
> pnls <- rowMeans(pnls, na.rm=TRUE)
> pnls <- xts::xts(pnls, datev)
> pnls <- pnls*sd(retew["/2014"])/sd(pnls["/2014"])
```




```
> # Combine with equal weight
> wealthv <- cbind(retew, pnls)
> colnames(wealthv) <- c("EqualWeight", "MaxSharpe")
> # Calculate the in-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv["/2014"], function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Calculate the out-of-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv["2015/"], function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Plot of cumulative portfolio returns
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Maximum Sharpe With Dimension Reduction") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyEvent(zoo::index(last(retis[, 1])), label="in-sample", stroke
+   dyLegend(width=300)
```

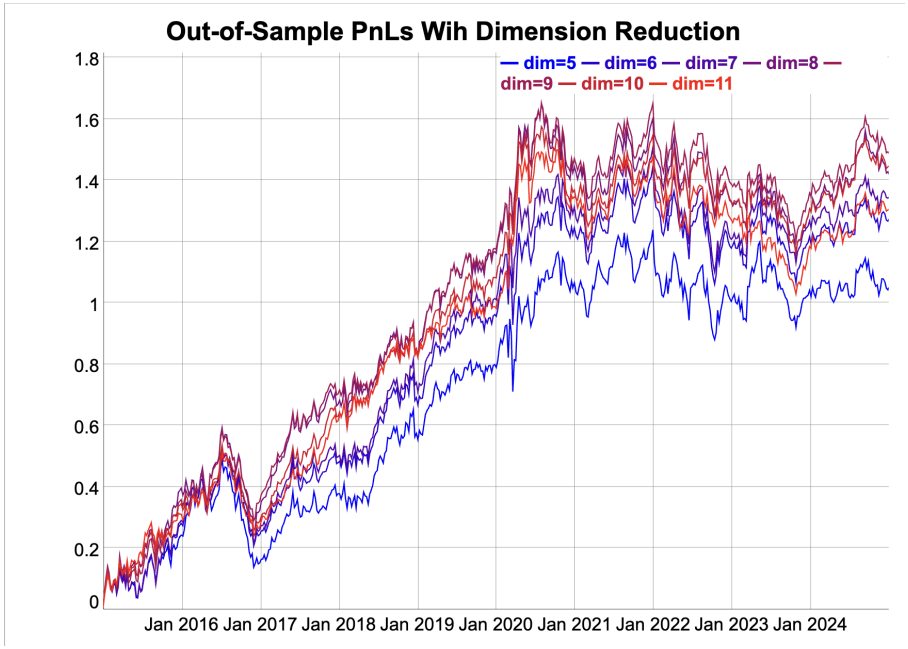
Optimal Dimension Reduction Out-of-Sample

The optimal out-of-sample dimension reduction parameter is `dimax=9`, which represents relatively weak dimension reduction.

The dependence of the out-of-sample strategy pnls on the *dimax* parameter reflects the *bias-variance tradeoff*.

If the *dimax* parameter is too small, it creates excessive bias, but if it's too large, it doesn't suppress the variance enough.

```
> # Perform loop over vector of dimension reduction parameters
> dimv <- seq(from=5, to=11)
> pnls <- mclapply(dimv, function(dimax) {
+   invred <- eigenvec[, 1:dimax] %*%
+     (t(eigenvec[, 1:dimax]) / eigenval[1:dimax])
+   ##  Calculate portfolio weights and PnLs
+   weightv <- invred %*% colmeanv
+   pnls <- HighFreq::mult_mat(weightv, retp)
+   pnls <- rowMeans(pnls, na.rm=TRUE)
+   pnls <- xts::xts(pnls, datev)
+   pnls*sd(retew["/2014"])/sd(pnls["/2014"])
+ }, mc.cores=ncores)  ##  end mclapply
> pnls <- do.call(cbind, pnls)
> colnames(pnls) <- paste0("dim=", dimv)
> profilev <- sapply(pnls["2015/"], sum)
> dimax <- dimv[which.max(profilev)]
```




```
> # Plot of cumulative portfolio returns
> colorv <- colorRampPalette(c("blue", "red"))(NCOL(pnls))
> endw <- rutils::calc_endpoints(pnls["2015/"], interval="weeks")
> dygraphs::dygraph(cumsum(pnls["2015/"])[endw],
+   main="Out-of-Sample PnLs Wih Dimension Reduction") %>%
+   dyOptions(colors=colorv, strokeWidth=1) %>%
+   dyLegend(show="always", width=300)
```

Maximum Sharpe Portfolio With Return Shrinkage

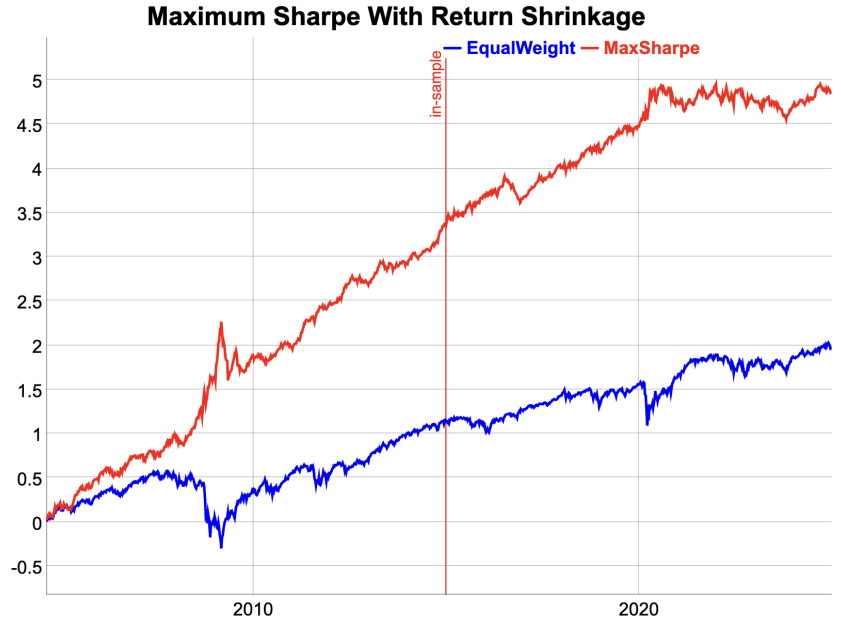
To further reduce the statistical noise, the individual returns r_i can be *shrunk* to the average portfolio returns $\bar{r}$:

$$r_i' = (1 - \alpha)\, r_i + \alpha\, \bar{r}$$

The parameter α is the *shrinkage* intensity, and it determines the strength of the *shrinkage* of individual returns to their mean.

If $\alpha = 0$ then there is no *shrinkage*, while if $\alpha = 1$ then all the returns are *shrunk* to their common mean: $r_i = \bar{r}$.

The optimal value of the *shrinkage* intensity α can be determined using *backtesting* (*cross-validation*).




```
> # Shrink the in-sample returns to their mean
> alphac <- 0.3
> retxm <- rowMeans(retx, na.rm=TRUE)
> retxis <- (1-alphac)*retx + alphac*retxm
> # Calculate portfolio weights and PnLs
> weightv <- drop(invred %*% colMeans(retxis, na.rm=TRUE))
> names(weightv) <- symbolv
> pnls <- HighFreq::mult_mat(weightv, retp)
> pnls <- rowMeans(pnls, na.rm=TRUE)
> pnls <- xts::xts(pnls, datev)
> pnls <- pnls*sd(retew["/2014"])/sd(pnls["/2014"])
```

```
> # Combine with equal weight
> wealthv <- cbind(retew, pnls)
> colnames(wealthv) <- c("EqualWeight", "MaxSharpe")
> # Calculate the in-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv["/2014"], function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Calculate the out-of-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv["2015/"], function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Plot of cumulative portfolio returns
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Maximum Sharpe With Return Shrinkage") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyEvent(zoo::index(last(retis[, 1])), label="in-sample", stroke
+   dyLegend(width=300)
```

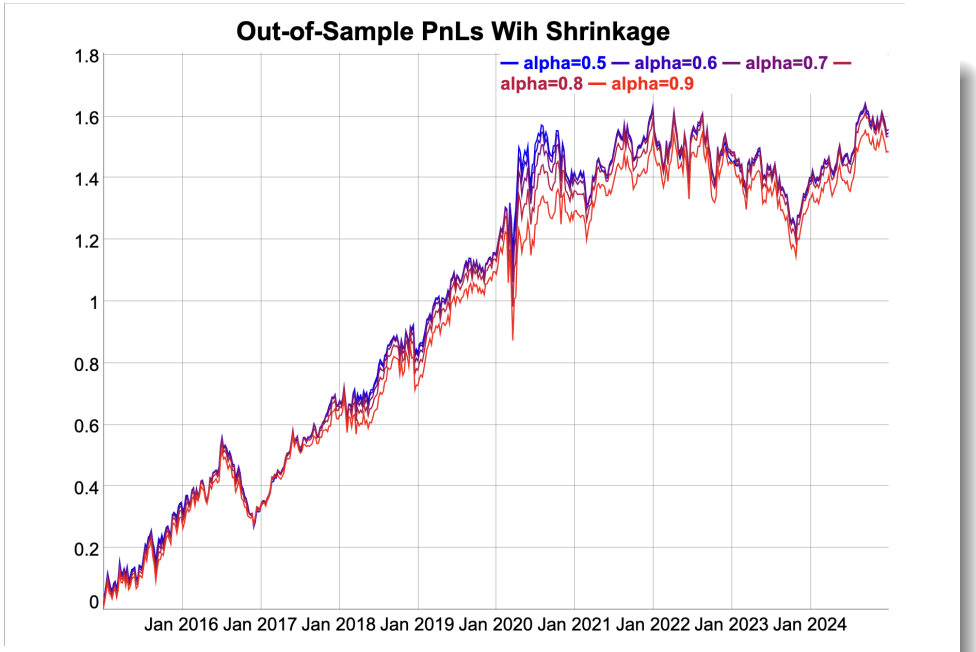
Optimal Shrinkage Parameter Out-of-Sample

The optimal out-of-sample return shrinkage parameter is equal to $\alpha = 0.6$, which represents moderate shrinkage.

The dependence of the out-of-sample strategy pnls on the α parameter reflects the *bias-variance tradeoff*.

If the α parameter is too large, it creates excessive bias, but if it's too small, it doesn't suppress the variance enough.

```
> # Perform loop over vector of shrinkage intensities
> alphav <- seq(from=0.5, to=0.9, by=0.1)
> pnls <- mclapply(alphav, function(alphac) {
+   retxis <- (1-alphac)*retx + alphac*retxm
+   weightv <- drop(invred %*% colMeans(retxis, na.rm=TRUE))
+   pnls <- HighFreq::mult_mat(weightv, retp)
+   pnls <- rowMeans(pnls, na.rm=TRUE)
+   pnls <- xts::xts(pnls, datev)
+   pnls*sd(retew["/2014"])/sd(pnls["/2014"])
+ }, mc.cores=ncores)  ##  end mclapply
> pnls <- do.call(cbind, pnls)
> colnames(pnls) <- paste0("alpha=", alphav)
> profilev <- sapply(pnls["2015/"], sum)
> alphac <- alphav[which.max(profilev)]
```




```
> # Plot of cumulative portfolio returns
> colorv <- colorRampPalette(c("blue", "red"))(NCOL(pnls))
> endw <- rutils::calc_endpoints(pnls["2015/"], interval="weeks")
> dygraphs::dygraph(cumsum(pnls["2015/"])[endw],
+   main="Out-of-Sample PnLs Wih Shrinkage") %>%
+   dyOptions(colors=colorv, strokeWidth=1) %>%
+   dyLegend(show="always", width=300)
```

Rolling Maximum Sharpe Stock Portfolio Strategy

A *rolling portfolio optimization* strategy consists of rebalancing a portfolio over the end points:

1. Calculate the maximum Sharpe ratio portfolio weights at each end point,
2. Apply the weights in the next interval and calculate the out-of-sample portfolio returns.

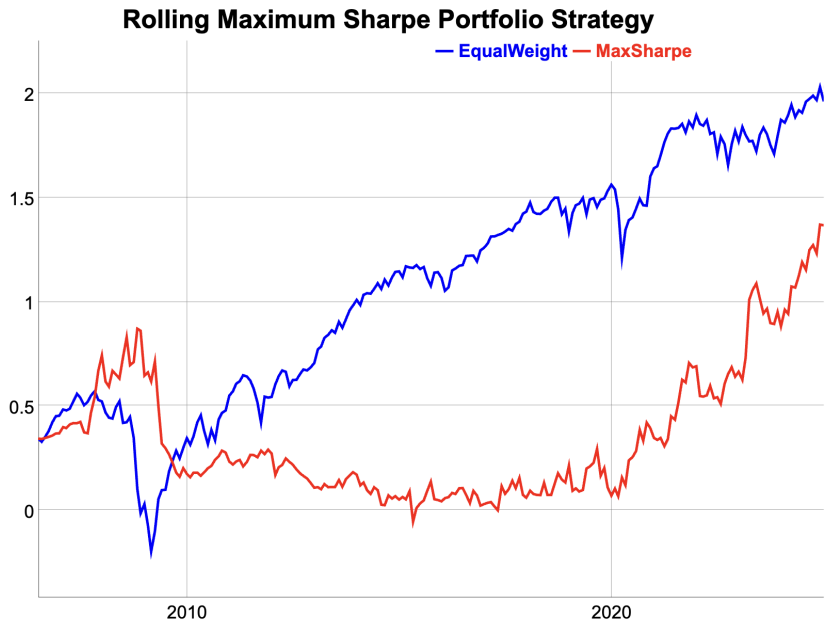
The strategy parameters are: the rebalancing frequency (annual, monthly, etc.), and the length of look-back interval.

```
> # Define monthly end points
> endd <- rutils::calc_endpoints(retp, interval="months")
> endd <- endd[endd > (nstocks+1)]
> npts <- NROW(endd) ; lookb <- 12
> startp <- c(rep_len(0, lookb), endd[1:(npts-lookb)])
> # !!! Perform parallel loop over end points - takes very long!!!
> library(parallel)  ##  Load package parallel
> ncores <- detectCores() - 1
> pnls <- mclapply(1:(npts-1), function(tday) {
+     ##  Subset the excess returns
+     retis <- retp[startp[tday]:endd[tday], ]
+     retis[is.na(retis)] <- 0
+     invreg <- MASS::ginv(cov(retis, use="pairwise.complete.obs"))
+     ##  Calculate the maximum Sharpe ratio portfolio weights
+     retm <- colMeans(retis, na.rm=TRUE)
+     weightv <- invreg %*% retm
+     ##  Zero weights if sparse data
+     zerov <- sapply(retis, function(x) (sum(x == 0) > 5))
+     weightv[zerov] <- 0
+     ##  Calculate in-sample portfolio returns
+     pnlis <- (retis %*% weightv)
+     ##  Scale the weights to the in-sample volatility
+     weightv <- weightv*sd(retm)/sd(pnlis)
+     ##  Calculate the out-of-sample portfolio returns
+     retos <- retp[(endd[tday]+1):endd[tday+1], ]
+     pnls <- HighFreq::mult_mat(weightv, retos)
+     pnls <- rowMeans(pnls, na.rm=TRUE)
+     xts::xts(pnls, zoo::index(retos))
+ }, mc.cores=ncores)  ##  end mclapply
> pnls <- rutils::do_call(rbind, pnls)
> pnls <- rbind(retew[paste0("/", start(pnls)-1)], pnls*sd(retew)/sd
```

Performance of Rolling Maximum Sharpe Strategy

The performance of the *rolling portfolio optimization* strategy can be improved by applying dimension reduction and return shrinkage.

```
> # Calculate the Sharpe and Sortino ratios
> wealthv <- cbind(retew, pnls)
> colnames(wealthv) <- c("EqualWeight", "MaxSharpe")
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
```




```
> # Plot cumulative strategy returns
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Rolling Maximum Sharpe Portfolio Strategy") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Fast Covariance Matrix Inverse Using *RcppArmadillo*

RcppArmadillo can be used to quickly calculate the reduced inverse of a covariance matrix.

```
> # Create random matrix of returns
> matv <- matrix(rnorm(300), nc=5)
> # Reduced inverse of covariance matrix
> dimax <- 3
> eigend <- eigen(covmat)
> invred <- eigend$vectors[, 1:dimax] %*%
+   (t(eigend$vectors[, 1:dimax]) / eigend$values[1:dimax])
> # Reduced inverse using RcppArmadillo
> invarma <- HighFreq::calc_inv(covmat, dimax)
> all.equal(invred, invarma)
> # Microbenchmark RcppArmadillo code
> library(microbenchmark)
> summary(microbenchmark(
+   rcode={eigend <- eigen(covmat)
+     eigend$vectors[, 1:dimax] %*%
+ (t(eigend$vectors[, 1:dimax]) / eigend$values[1:dimax])
+   },
+   rcpp=calc_inv(covmat, dimax),
+   times=10))[, c(1, 4, 5)]  ##  end microbenchmark summary
```

```
arma::mat calc_inv(const arma::mat& matv,
                   arma::uword dimax = 0, // Max number
                   double eigen_thresh = 0.01) { // Thre

  if (dimax == 0) {
    // Calculate the inverse using arma::pinv()
    return arma::pinv(tseries, eigen_thresh);
  } else {
    // Calculate the reduced inverse using SVD decomposi

    // Allocate SVD
    arma::vec svdval;
    arma::mat svdu, svdv;

    // Calculate the SVD
    arma::svd(svdu, svdval, svdv, tseries);

    // Subset the SVD
    dimax = dimax - 1;
    // For no regularization: dimax = tseries.n_cols
    svdu = svdu.cols(0, dimax);
    svdv = svdv.cols(0, dimax);
    svdval = svdval.subvec(0, dimax);

    // Calculate the inverse from the SVD
    return svdv*arma::diagmat(1/svdval)*svdu.t();

  }  // end if

}  // end calc_inv
```

Portfolio Optimization Using *RcppArmadillo*

Fast portfolio optimization using matrix algebra can be implemented using *RcppArmadillo*.

```
arma::vec calc_weights(const arma::mat& returns, // Asset returns
                       Rcpp::List controll) { // List of portfolio optimization parameters

  // Unpack the control list of portfolio optimization parameters
  // Type of portfolio optimization model
  std::string method = Rcpp::as<std::string>(controll["method"]);
  // Threshold level for discarding small singular values
  double eigen_thresh = Rcpp::as<double>(controll["eigen_thresh"]);
  // Dimension reduction
  arma::uword dimax = Rcpp::as<int>(controll["dimax"]);
  // Confidence level for calculating the quantiles of returns
  double confl = Rcpp::as<double>(controll["confl"]);
  // Shrinkage intensity of returns
  double alpha = Rcpp::as<double>(controll["alpha"]);
  // Should the weights be ranked?
  bool rankw = Rcpp::as<int>(controll["rankw"]);
  // Should the weights be centered?
  bool centerw = Rcpp::as<int>(controll["centerw"]);
  // Method for scaling the weights
  std::string scalew = Rcpp::as<std::string>(controll["scalew"]);
  // Volatility target for scaling the weights
  double vol_target = Rcpp::as<double>(controll["vol_target"]);

  // Initialize the variables
  arma::uword ncols = returns.n_cols;
  arma::vec weightv(ncols, fill::zeros);
  // If no regularization then set dimax to ncols
  if (dimax == 0)  dimax = ncols;
  // Calculate the covariance matrix
  arma::mat covmat = calc_covar(returns);

  // Apply different calculation methods for the weights
  switch(calc_method(method)) {
  case methodenum::maxsharpe: {
    // Mean returns of columns
```

Strategy Backtesting Using *RcppArmadillo*

Fast backtesting of strategies can be implemented using *RcppArmadillo*.

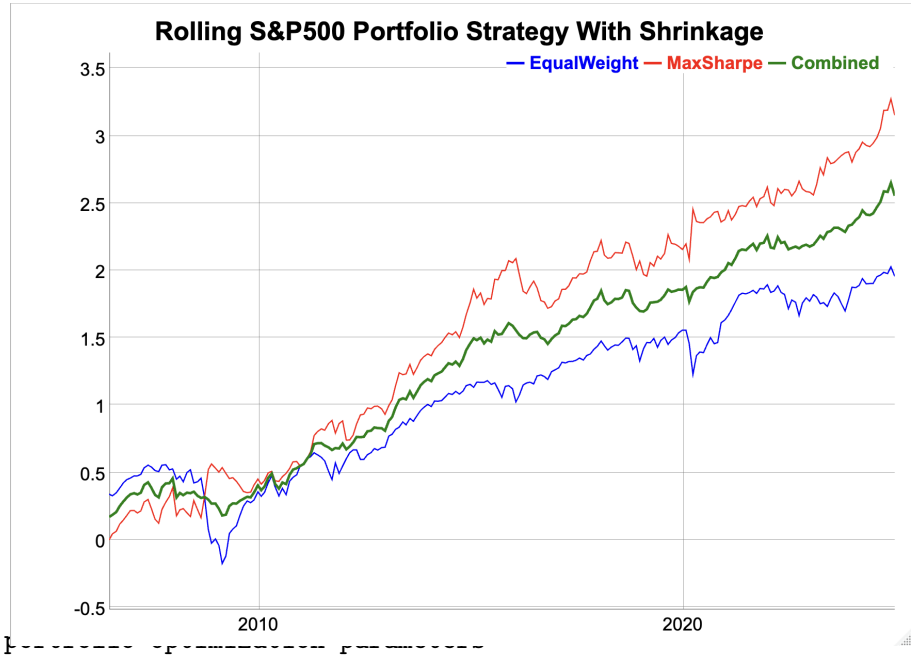
```
arma::mat roll_portf(const arma::mat& retx, // Asset excess returns
                     const arma::mat& retp, // Asset returns
                     Rcpp::List controll, // List of portfolio optimization model parameters
                     arma::uvec startp, // Start points
                     arma::uvec endp, // End points
                     double lambdaf = 0.0, // Decay factor for averaging the portfolio weights
                     double coeff = 1.0, // Multiplier of strategy returns
                     double bidask = 0.0) { // The bid-ask spread

  double lambda1 = 1-lambdaf;
  arma::uword nweights = retp.n_cols;
  arma::vec weightv(nweights, fill::zeros);
  arma::vec weights_past = arma::ones(nweights)/std::sqrt(nweights);
  arma::mat pnls = arma::zeros(retp.n_rows, 1);

  // Perform loop over the end points
  for (arma::uword it = 1; it < endp.size(); it++) {
    // cout << "it: " << it << endl;
    // Calculate the portfolio weights
    weightv = coeff*calc_weights(retx.rows(startp(it-1), endp(it-1)), controll);
    // Calculate the weights as the weighted sum with past weights
    weightv = lambda1*weightv + lambdaf*weights_past;
    // Calculate out-of-sample returns
    pnls.rows(endp(it-1)+1, endp(it)) = retp.rows(endp(it-1)+1, endp(it))*weightv;
    // Add transaction costs
    pnls.row(endp(it-1)+1) -= bidask*sum(abs(weightv - weights_past))/2;
    // Copy the weights
    weights_past = weightv;
  }  // end for

  // Return the strategy pnls
  return pnls;

}  // end roll_portf
```

Rolling Portfolio Strategy With Dimension Reduction and Shrinkage

The performance of the *rolling portfolio optimization* strategy can be improved by applying dimension reduction and return shrinkage.

The *rolling portfolio* strategy has a higher *Sharpe* ratio and also higher absolute returns than the equal weight portfolio.

The backtest simulation can be performed very quickly using C++ code and package *RcppArmadillo*.




```
> # Shift the end points to C++ convention
> endd <- (endd - 1)
> endd[endd < 0] <- 0
> startp <- (startp - 1)
> startp[startp < 0] <- 0
> # Specify dimension reduction and return shrinkage using list of
> dimax <- 9
> alphac <- 0.8
> controll <- HighFreq::param_portf(method="maxsharpe",
+   dimax=dimax, alpha=alphac)
> # Perform backtest in Rcpp - takes very long!!!
> retx <- (retp - raterf)
> pnls <- HighFreq::roll_portf(retx=retx, retp=retp,
+   startp=startp, endd=endd, controll=controll)
> pnls <- pnls*sd(retew)/sd(pnls)
```

```
> # Calculate the out-of-sample Sharpe and Sortino ratios
> wealthv <- cbind(retew, pnls, (pnls + retew)/2)
> colnames(wealthv) <- c("EqualWeight", "MaxSharpe", "Combined")
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Plot cumulative strategy returns
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Rolling S&P500 Portfolio Strategy With Shrinkage") %>%
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dySeries(name="Combined", label="Combined", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Optimal Dimension Reduction Parameter

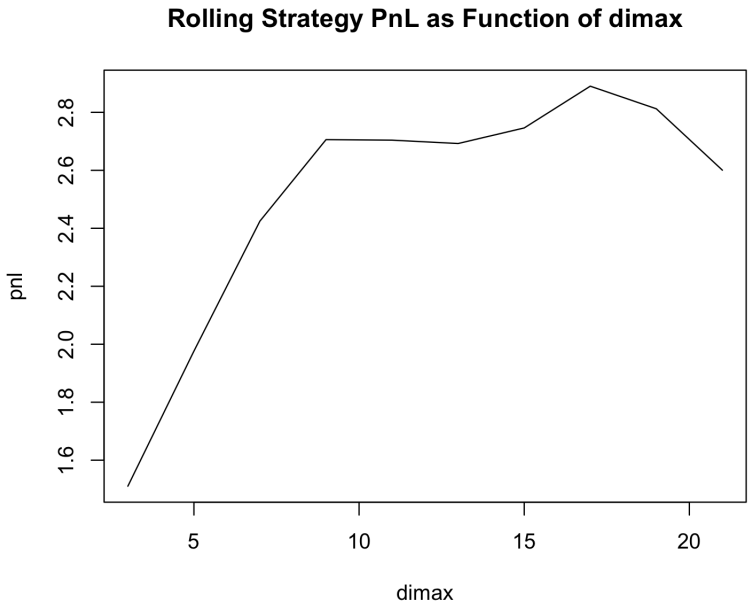
The optimal value of the dimension reduction parameter *dimax* can be determined using *backtesting*.

The optimal dimension reduction parameter for this portfolio of stocks is equal to `dimax=9`, which represents relatively weak dimension reduction.

The dependence of the out-of-sample strategy pnls on the *dimax* parameter reflects the *bias-variance tradeoff*.

If the *dimax* parameter is too small, it creates excessive bias, but if it's too large, it doesn't suppress the variance enough.

```
> # Perform backtest over vector of dimension reduction parameters
> dimv <- seq(from=3, to=21, by=2)
> pnls <- mclapply(dimv, function(dimax) {
+   controll <- HighFreq::param_portf(method="maxsharpe",
+     dimax=dimax, alpha=alphac)
+   HighFreq::roll_portf(retx=retx, retp=retp,
+     startp=startp, endd=endd, controll=controll)
+ }, mc.cores=ncores)  ##  end mclapply
> profilev <- sapply(pnls, sum)
> dimax <- dimv[which.max(profilev)]
```




```
> # Plot of rolling strategy PnL as a function of dimax
> plot(x=dimv, y=profilev, t="l", xlab="dimax", ylab="pnl",
+   main="Rolling Strategy PnL as Function of dimax")
```

Optimal Shrinkage Parameter

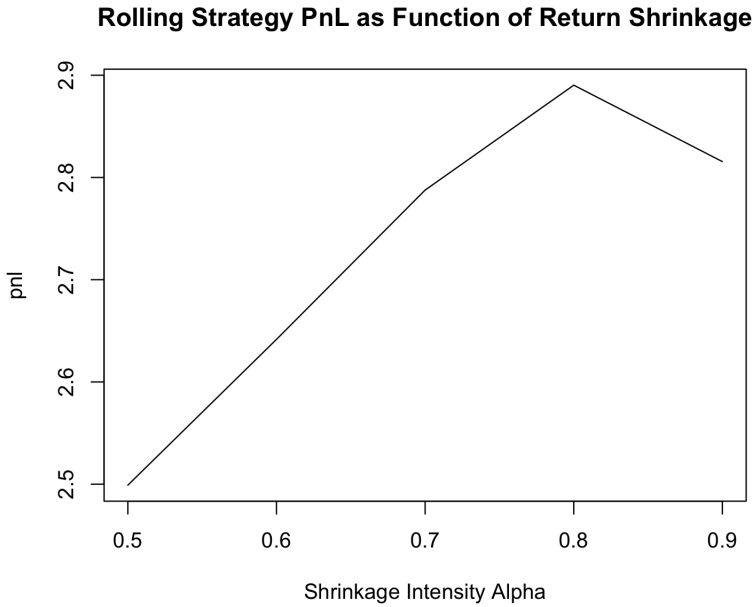
The optimal value of the return shrinkage intensity parameter α can be determined using *backtesting*.

The best return shrinkage parameter for this portfolio of stocks is equal to $\alpha = 0.8$, which represents strong shrinkage.

The dependence of the out-of-sample strategy pnls on the α parameter reflects the *bias-variance tradeoff*.

If the α parameter is too large, it creates excessive bias, but if it's too small, it doesn't suppress the variance enough.

```
> # Perform backtest over vector of shrinkage intensities
> alphav <- seq(from=0.5, to=0.9, by=0.1)
> pnls <- mclapply(alphav, function(alphac) {
+   controll <- HighFreq::param_portf(method="maxsharpe",
+       dimax=dimax, alpha=alphac)
+   HighFreq::roll_portf(retx=retx, retp=retp,
+       startp=startp, endd=endd, controll=controll)
+ }, mc.cores=ncores)  ##  end mclapply
> profilev <- sapply(pnls, sum)
> alphac <- alphav[which.max(profilev)]
```




```
> # Plot of rolling strategy PnL as a function of shrinkage intensit
> plot(x=alphav, y=profilev, t="l",
+   main="Rolling Strategy PnL as Function of Return Shrinkage",
+   xlab="Shrinkage Intensity Alpha", ylab="pnl")
```

Optimal Look-back Interval

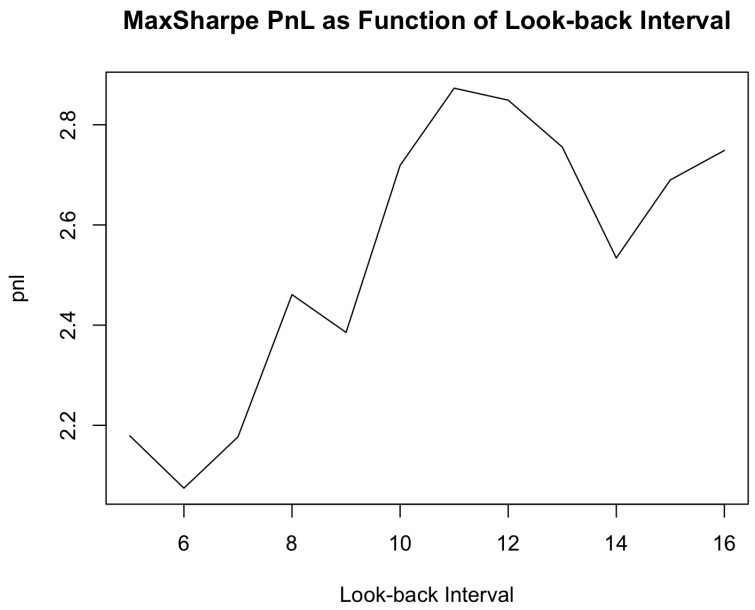
The optimal length of the look-back interval can be determined using *backtesting*.

The optimal length of the look-back interval for this portfolio of stocks is equal to `lookb=10` months, which roughly agrees with the research literature on momentum strategies.

The dependence on the length of the *look-back interval* is an example of the *bias-variance tradeoff*.

If the *look-back interval* is too long, then the data has large *bias* because the distant past may have little relevance to today.

But if the *look-back interval* is too short, then there's not enough data, and estimates will have high *variance*.



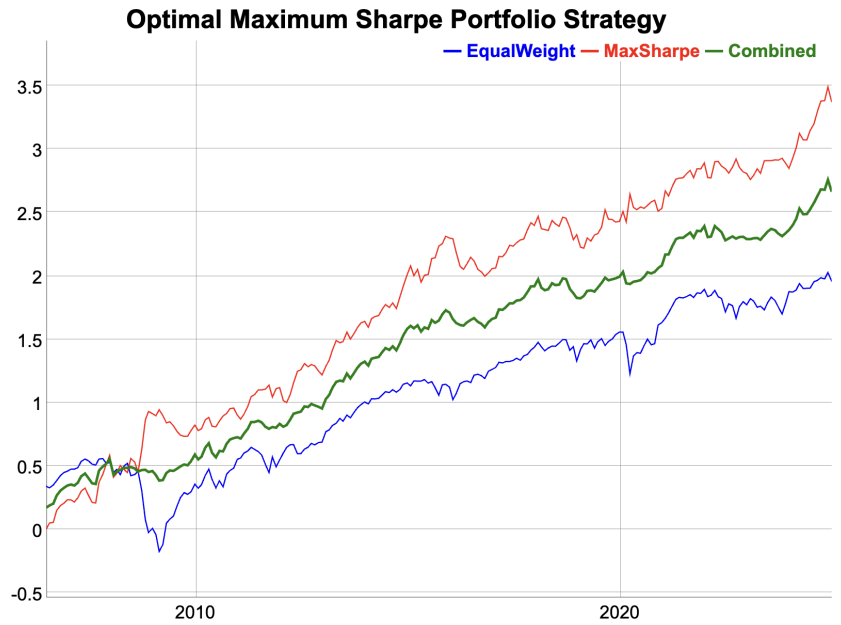

```
> # Create list of model parameters
> controll <- HighFreq::param_portf(method="maxsharpe",
+       dimax=dimax, alpha=alphac)
> # Perform backtest over look-backs
> lookbv <- seq(from=5, to=16, by=1)
> pnls <- mclapply(lookbv, function(lookb) {
+   startp <- c(rep_len(0, lookb), endd[1:(npts-lookb)])
+   startp <- (startp - 1) ; startp[startp < 0] <- 0
+   HighFreq::roll_portf(retx=retx, retp=retp,
+     startp=startp, endd=endd, controll=controll)
+ }, mc.cores=ncores)  ##  end mclapply
> profilev <- sapply(pnls, sum)
> lookb <- lookbv[which.max(profilev)]
```

```
> # Plot of rolling strategy PnL as a function of look-back interval
> plot(x=lookbv, y=profilev, t="l", main="MaxSharpe PnL as Function
+   xlab="Look-back Interval", ylab="pnl")
```

Optimal Portfolio Optimization Strategy

The optimal *rolling portfolio optimization* strategy, with dimension reduction and return shrinkage, has both a higher *Sharpe* ratio and higher absolute returns than the equal weight portfolio.

Combining the optimal rolling portfolio strategy with the equal weight portfolio results in an even higher *Sharpe* ratio.




```
> # Calculate the out-of-sample Sharpe and Sortino ratios
> pnls <- pnls[[whichmax]]
> pnls <- pnls*sd(retew)/sd(pnls)
> wealthv <- cbind(retew, pnls, (pnls + retew)/2)
> colnames(wealthv) <- c("EqualWeight", "MaxSharpe", "Combined")
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Dygraph the cumulative wealth
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Optimal Maximum Sharpe Portfolio Strategy") %>%
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dySeries(name="Combined", label="Combined", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Package *Rcpp* for Calling C++ Programs from R

The package *Rcpp* allows calling C++ functions from R, by compiling the C++ code and creating R functions.

Rcpp functions are R functions that were compiled from C++ code using package *Rcpp*.

Rcpp functions are much faster than code written in R, so they're suitable for large numerical calculations.

The package *Rcpp* relies on *Rtools* for compiling the C++ code:

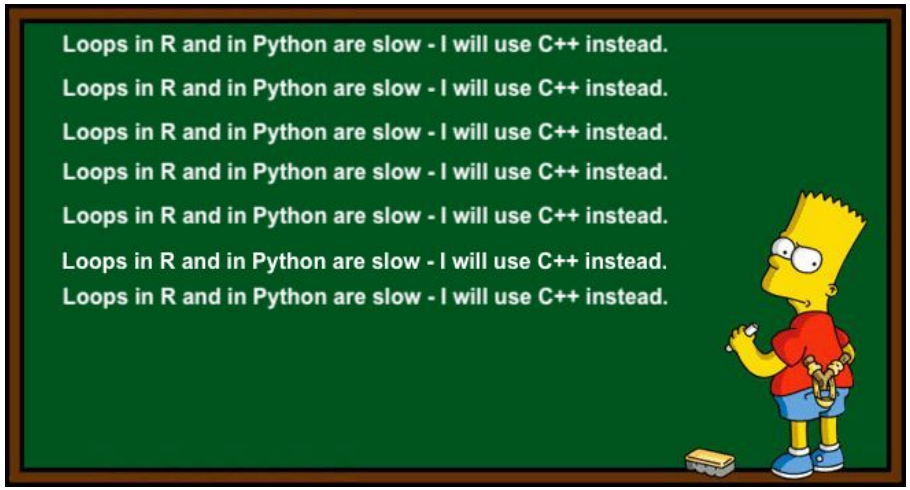
https://cran.r-project.org/bin/windows/Rtools/

You can learn more about the package *Rcpp* here:

http://adv-r.had.co.nz/Rcpp.html

http://www.rcpp.org/

http://gallery.rcpp.org/




```
> # Verify that Rtools or XCode are working properly:
> devtools::find_rtools()  ##  Under Windows
> devtools::has_devel()
> # Install the packages Rcpp and RcppArmadillo
> install.packages(c("Rcpp", "RcppArmadillo"))
> # Load package Rcpp
> library(Rcpp)
> # Get documentation for package Rcpp
> # Get short description
> packageDescription("Rcpp")
> # Load help page
> help(package="Rcpp")
> # List all datasets in "Rcpp"
> data(package="Rcpp")
> # List all objects in "Rcpp"
> ls("package:Rcpp")
> # Remove Rcpp from search path
> detach("package:Rcpp")
```

Function cppFunction() for Compiling C++ code

The function cppFunction() compiles C++ code into an R function.

The function cppFunction() creates an R function only for the current R session, and it must be recompiled for every new R session.

The function sourceCpp() compiles C++ code contained in a file into R functions.

```
> # Define Rcpp function
> Rcpp::cppFunction("
+   int times_two(int x)
+     { return 2 * x;}
+   ")  ##  end cppFunction
> # Run Rcpp function
> times_two(3)
> # Source Rcpp functions from file
> Rcpp::sourceCpp(file="/Users/jerzy/Develop/lecture_slides/scripts/
> # Multiply two numbers
> mult_rcpp(2, 3)
> mult_rcpp(1:3, 6:4)
> # Multiply two vectors
> mult_vec_rcpp(2, 3)
> mult_vec_rcpp(1:3, 6:4)
```

Performing Loops in *Rcpp Sugar*

Loops written in *Rcpp* can be two orders of magnitude faster than loops in R!

Rcpp Sugar allows using R-style vectorized syntax in *Rcpp* code.

```
> # Define Rcpp function with loop
> Rcpp::cppFunction("
+ double inner_mult(NumericVector x, NumericVector y) {
+ int xsize = x.size();
+ int ysize = y.size();
+ if (xsize != ysize) {
+     return 0;
+   } else {
+     double total = 0;
+     for(int i = 0; i < xsize; ++i) {
+ total += x[i] * y[i];
+   }
+   return total;
+   }
+ }")  ##  end cppFunction
> # Run Rcpp function
> inner_mult(1:3, 6:4)
> inner_mult(1:3, 6:3)
> # Define Rcpp Sugar function with loop
> Rcpp::cppFunction("
+ double inner_sugar(NumericVector x, NumericVector y) {
+   return sum(x * y);
+ }")  ##  end cppFunction
> # Run Rcpp Sugar function
> inner_sugar(1:3, 6:4)
> inner_sugar(1:3, 6:3)
```

```
> # Define R function with loop
> inner_multr <- function(x, y) {
+     sumv <- 0
+     for(i in 1:NROW(x)) {
+ sumv <- sumv + x[i] * y[i]
+     }
+     sumv
+ }  ##  end inner_multr
> # Run R function
> inner_multr(1:3, 6:4)
> inner_multr(1:3, 6:3)
> # Compare speed of Rcpp and R
> library(microbenchmark)
> summary(microbenchmark(
+   rcode=inner_multr(1:10000, 1:10000),
+   innerp=1:10000 %*% 1:10000,
+   Rcpp=inner_mult(1:10000, 1:10000),
+   sugar=inner_sugar(1:10000, 1:10000),
+   times=10))[, c(1, 4, 5)]
```

Simulating Ornstein-Uhlenbeck Process Using *Rcpp*

Simulating the Ornstein-Uhlenbeck Process in *Rcpp* is about 30 times faster than in R!

```
> # Define Ornstein-Uhlenbeck function in R
> sim_our <- function(nrows=1000, priceq=5.0,
+               volat=0.01, theta=0.01) {
+   retp <- numeric(nrows)
+   pricev <- numeric(nrows)
+   pricev[1] <- priceq
+   for (i in 2:nrows) {
+     retp[i] <- theta*(priceq - pricev[i-1]) + volat*rnorm(1)
+     pricev[i] <- pricev[i-1] + retp[i]
+   }  ##  end for
+   pricev
+ }  ##  end sim_our
> # Simulate Ornstein-Uhlenbeck process in R
> priceq <- 5.0; sigmav <- 0.01
> thetav <- 0.01; nrows <- 1000
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")  ##
> ousim <- sim_our(nrows, priceq=priceq, volat=sigmav, theta=theta
```

```
> # Define Ornstein-Uhlenbeck function in Rcpp
> Rcpp::cppFunction("
+ NumericVector sim_oucpp(double priceq,
+                   double volat,
+                   double thetav,
+                   NumericVector innov) {
+   int nrows = innov.size();
+   NumericVector pricev(nrows);
+   NumericVector retv(nrows);
+   pricev[0] = priceq;
+   for (int it = 1; it < nrows; it++) {
+     retv[it] = thetav*(priceq - pricev[it-1]) + volat*innov[it-1];
+     pricev[it] = pricev[it-1] + retv[it];
+   }  // end for
+   return pricev;
+ }")  ##  end cppFunction
> # Simulate Ornstein-Uhlenbeck process in Rcpp
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")  ##  R
> oucpp <- sim_oucpp(priceq=priceq,
+   volat=sigmav, theta=thetav, innov=rnorm(nrows))
> all.equal(ousim, oucpp)
> # Compare speed of Rcpp and R
> library(microbenchmark)
> summary(microbenchmark(
+   rcode=sim_our(nrows, priceq=priceq, volat=sigmav, theta=thetav),
+   Rcpp=sim_oucpp(priceq=priceq, volat=sigmav, theta=thetav, innov=
+   times=10))[, c(1, 4, 5)]
```

Rcpp Attributes

Rcpp attributes are instructions for the C++ compiler, embedded in the *Rcpp* code as C++ comments, and preceded by the "//" symbol.

The `Rcpp::depends` attribute specifies additional C++ library dependencies.

The `Rcpp::export` attribute specifies that a function should be exported to R, where it can be called as an R function.

Only functions which are preceded by the `Rcpp::export` attribute are exported to R.

The function `sourceCpp()` compiles C++ code contained in a file into R functions.

```
// Rcpp header with information for C++ compiler
#include <Rcpp.h> // include Rcpp C++ header files
using namespace Rcpp; // use Rcpp C++ namespace

// The function sim_oucpp() simulates an Ornstein-Uhlenb
// export the function roll_maxmin() to R
// [[Rcpp::export]]
NumericVector sim_oucpp(double priceq,
                        double volat,
                        double thetav,
                        NumericVector innov) {
  int(nrows = innov.size();
  NumericVector pricev*nrows);
  NumericVector retp*nrows);
  pricev[0] = priceq;
  for (int it = 1; it < nrows; it++) {
    retp[it] = thetav*(priceq - pricev[it-1]) + volat*in
    pricev[it] = pricev[it-1] + retp[it];
  }  // end for
  return pricev;
}  // end sim_oucpp
```

```
> # Source Rcpp function for Ornstein-Uhlenbeck process from file
> Rcpp::sourceCpp(file="/Users/jerzy/Develop/lecture_slides/scripts/
> # Simulate Ornstein-Uhlenbeck process in Rcpp
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")  ##  Reset random numbers
> oucpp <- sim_oucpp(priceq=priceq,
+   volat=sigmav,
+   theta=thetav,
+   innov=rnorm(nrows))
> all.equal(ousim, oucpp)
> # Compare speed of Rcpp and R
> library(microbenchmark)
> summary(microbenchmark(
+   rcode=sim_our(nrows, priceq=priceq, volat=sigmav, theta=thetav),
+   Rcpp=sim_oucpp(priceq=priceq, volat=sigmav, theta=thetav, innov=rnorm(nrows)),
+   times=10))[, c(1, 4, 5)]
```

Generating Random Numbers Using Logistic Map in *Rcpp*

The *logistic map* in *Rcpp* is about seven times faster than the loop in R, and even slightly faster than the standard `runif()` function in R!

```
> # Calculate uniformly distributed pseudo-random sequence
> unifun <- function(seedv, nrows=10) {
+   datav <- numeric(nrows)
+   datav[1] <- seedv
+   for (i in 2:nrows) {
+     datav[i] <- 4*datav[i-1]*(1-datav[i-1])
+   }  ##  end for
+   acos(1-2*datav)/pi
+ }  ##  end unifun
>
> # Source Rcpp functions from file
> Rcpp::sourceCpp(file="/Users/jerzy/Develop/lecture_slides/scripts,
> # Microbenchmark Rcpp code
> library(microbenchmark)
> summary(microbenchmark(
+   rcode=runif(1e5),
+   rloop=unifun(0.3, 1e5),
+   Rcpp=unifuncpp(0.3, 1e5),
+   times=10))[, c(1, 4, 5)]
```

```
// Rcpp header with information for C++ compiler
#include <Rcpp.h> // include Rcpp C++ header files
using namespace Rcpp; // use Rcpp C++ namespace

// This is a simple example of exporting a C++ function
// You can source this function into an R session using
// function Rcpp::sourceCpp()
// (or via the Source button on the editor toolbar).
// Learn more about Rcpp at:
//
//   http://www.rcpp.org/
//   http://adv-r.had.co.nz/Rcpp.html
//   http://gallery.rcpp.org/

// function unifun() produces a vector of
// uniformly distributed pseudo-random numbers
// [[Rcpp::export]]
NumericVector unifuncpp(double seedv, int(nrows) {
// define pi
static const double pi = 3.14159265;
// allocate output vector
  NumericVector datav(nrows);
// initialize output vector
  datav[0] = seedv;
// perform loop
  for (int i=1; i < nrows; ++i) {
    datav[i] = 4*datav[i-1]*(1-datav[i-1]);
  }  // end for
// rescale output vector and return it
  return acos(1-2*datav)/pi;
}
```

Package *RcppArmadillo* for Fast Linear Algebra

The package *RcppArmadillo* allows calling from R the high-level *Armadillo* C++ linear algebra library.

Armadillo provides ease of use and speed, with syntax similar to *Matlab*.

RcppArmadillo functions are often faster than even compiled R functions, because they use better optimized C++ code:
http://arma.sourceforge.net/speed.html

You can learn more about *RcppArmadillo*:
http://arma.sourceforge.net/
http://dirk.eddelbuettel.com/code/rcpp.armadillo.html
https://cran.r-project.org/web/packages/\emph{RcppArmadillo}/index.html
https://github.com/RcppCore/\emph{RcppArmadillo}

```
> library(RcppArmadillo)
> # Source Rcpp functions from file
> Rcpp::sourceCpp(file="/Users/jerzy/Develop/lecture_slides/scripts
> vec1 <- runif(1e5)
> vec2 <- runif(1e5)
> inner_vec(vec1, vec2)
> vec1 %*% vec2
```

```
// Rcpp header with information for C++ compiler
#include <RcppArmadillo.h>
using namespace Rcpp;
using namespace arma;
// [[Rcpp::depends(RcppArmadillo)]]

// The function inner_vec() calculates the inner (dot) p
// It uses \emph{RcppArmadillo}.
//' @export
// [[Rcpp::export]]
double inner_vec(arma::vec vec1, arma::vec vec2) {
  return arma::dot(vec1, vec2);
}  // end inner_vec

// The function inner_mat() calculates the inner (dot) p
// with two vectors.
// It accepts pointers to the matrix and vectors, and re
// It uses \emph{RcppArmadillo}.
//' @export
// [[Rcpp::export]]
double inner_mat(const arma::vec& vecv2, const arma::mat
  return arma::as_scalar(trans(vecv2) * (matv * vecv1));
}  // end inner_mat
```

```
> # Microbenchmark \emph{RcppArmadillo} code
> summary(microbenchmark(
+   rcpp = inner_vec(vec1, vec2),
+   rcode = (vec1 %*% vec2),
+   times=100))[, c(1, 4, 5)]  ##  end microbenchmark summary
> # Microbenchmark shows:
> # inner_vec() is several times faster than %*%, especially for lo
> #     expr     mean   median
> # 1 inner_vec 110.7067 110.4530
> # 2 rcode 585.5127 591.3575
```

Simulating *ARIMA* Processes Using *RcppArmadillo*

ARIMA processes can be simulated using *RcppArmadillo* even faster than by using the function `filter()`.

```
> # Source Rcpp functions from file
> Rcpp::sourceCpp(file="/Users/jerzy/Develop/lecture_slides/scripts/
> # Define AR(2) coefficients
> coeff <- c(0.9, 0.09)
> nrows <- 1e4
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> innov <- rnorm(nrows)
> # Simulate ARIMA using filter()
> arimar <- filter(x=innov, filter=coeff, method="recursive")
> # Simulate ARIMA using sim_ar()
> innov <- matrix(innov)
> coeff <- matrix(coeff)
> arimav <- sim_ar(coeff, innov)
> all.equal(drop(arimav), as.numeric(arimar))
> # Microbenchmark \emph{RcppArmadillo} code
> summary(microbenchmark(
+   rcpp = sim_ar(coeff, innov),
+   filter = filter(x=innov, filter=coeff, method="recursive"),
+   times=100))[, c(1, 4, 5)]  ##  end microbenchmark summary
```

```
// Rcpp header with information for C++ compiler
#include <RcppArmadillo.h> // include C++ header file fr
using namespace arma; // use C++ namespace from Armadill
// declare dependency on RcppArmadillo
// [[Rcpp::depends(RcppArmadillo)]]

//' @export
// [[Rcpp::export]]
arma::vec sim_ar(const arma::vec& innov, const arma::vec
  uword nrows = innov.n_elem;
  uword lookb = coeff.n_elem;
  arma::vec arimav[nrows);

  // startup period
  arimav(0) = innov(0);
  arimav(1) = innov(1) + coeff(lookb-1) * arimav(0);
  for (uword it = 2; it < lookb-1; it++) {
    arimav(it) = innov(it) + arma::dot(coeff.subvec(lookb
  }  // end for

  // remaining periods
  for (uword it = lookb; it < nrows; it++) {
    arimav(it) = innov(it) + arma::dot(coeff, arimav.sub
  }  // end for

  return arimav;
}  // end sim_arima
```

Fast Matrix Algebra Using *RcppArmadillo*

RcppArmadillo functions can be made even faster by operating on pointers to matrices and performing calculations in place, without copying large matrices.

RcppArmadillo functions can be compiled using the same *Rtools* as those for *Rcpp* functions:
https://cran.r-project.org/bin/windows/Rtools/

```
> library(RcppArmadillo)
> # Source Rcpp functions from file
> Rcpp::sourceCpp(file="/Users/jerzy/Develop/lecture_slides/scripts/
> matv <- matrix(runif(1e5), nc=1e3)
> # Center matrix columns using apply()
> matd <- apply(matv, 2, function(x) (x-mean(x)))
> # Center matrix columns in place using Rcpp demeanr()
> demeanr(matv)
> all.equal(matd, matv)
> # Microbenchmark \emph{RcppArmadillo} code
> library(microbenchmark)
> summary(microbenchmark(
+   rcode = (apply(matv, 2, mean)),
+   rcpp = demeanr(matv),
+   times=100))[, c(1, 4, 5)]  ##  end microbenchmark summary
> # Perform matrix inversion
> # Create random positive semi-definite matrix
> matv <- matrix(runif(25), nc=5)
> matv <- t(matv) %*% matv
> # Invert the matrix
> matrixinv <- solve(matv)
> inv_mat(matv)
> all.equal(matrixinv, matv)
> # Microbenchmark \emph{RcppArmadillo} code
> summary(microbenchmark(
+   rcode = solve(matv),
+   rcpp = inv_mat(matv),
+   times=100))[, c(1, 4, 5)]  ##  end microbenchmark summary
```

```
// Rcpp header with information for C++ compiler
#include <RcppArmadillo.h> // include C++ header file fr
using namespace arma; // use C++ namespace from Armadill
// declare dependency on RcppArmadillo
// [[Rcpp::depends(RcppArmadillo)]]

// Examples of \emph{RcppArmadillo} functions below

// The function demeanr() calculates a matrix with cente
// It accepts a pointer to a matrix and operates on the
// It returns the number of columns of the input matrix.
// It uses \emph{RcppArmadillo}.
//' @export
// [[Rcpp::export]]
int demeanr(arma::mat& matv) {
  for (uword i = 0; i < matv.n_cols; i++) {
    matv.col(i) -= arma::mean(matv.col(i));
  }  // end for
  return matv.n_cols;
}  // end demeanr

// The function inv_mat() calculates the inverse of symm
// definite matrix.
// It accepts a pointer to a matrix and operates on the
// It returns the number of columns of the input matrix.
// It uses \emph{RcppArmadillo}.
//' @export
// [[Rcpp::export]]
double inv_mat(arma::mat& matv) {
  matv = arma::inv_sympd(matv);
  return matv.n_cols;
}  // end inv_mat
```

Fast Correlation Matrix Inverse Using *RcppArmadillo*

RcppArmadillo can be used to quickly calculate the reduced inverse of correlation matrices.

```
> library(RcppArmadillo)
> # Source Rcpp functions from file
> Rcpp::sourceCpp("/Users/jerzy/Develop/lecture_slides/scripts/HighI
> # Calculate matrix of random returns
> matv <- matrix(rnorm(300), nc=5)
> # Reduced inverse of correlation matrix
> dimax <- 4
> cormat <- cor(matv)
> eigend <- eigen(cormat)
> invmat <- eigend$vectors[, 1:dimax] %*%
+   (t(eigend$vectors[, 1:dimax]) / eigend$values[1:dimax])
> # Reduced inverse using \emph{RcppArmadillo}
> invarma <- calc_inv(cormat, dimax=dimax)
> all.equal(invmat, invarma)
> # Microbenchmark \emph{RcppArmadillo} code
> library(microbenchmark)
> summary(microbenchmark(
+   rcode = {eigend <- eigen(cormat)
+ eigend$vectors[, 1:dimax] %*% (t(eigend$vectors[, 1:dimax]) / eige
+   rcpp = calc_inv(cormat, dimax=dimax),
+   times=100))[, c(1, 4, 5)]  ##  end microbenchmark summary
```

```
// Rcpp header with information for C++ compiler
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>
// include Rcpp C++ header files
using namespace stdev;
using namespace Rcpp; // use Rcpp C++ namespace
using namespace arma;

//' @export
// [[Rcpp::export]]
arma::mat calc_inv(const arma::mat& matv,
                   arma::uword dimax = 0, // Max number
                   double eigen_thresh = 0.01) { // Thre

  // Allocate SVD variables
  arma::vec svdval;  // Singular values
  arma::mat svdu, svdv;  // Singular matrices
  // Calculate the SVD
  arma::svd(svdu, svdval, svdv, tseries);
  // Calculate the number of non-small singular values
  arma::uword svdnum = arma::sum(svdval > eigen_thresh*a

  // If no regularization then set dimax to (svdnum - 1)
  if (dimax == 0) {
    // Set dimax
    dimax = svdnum - 1;
  } else {
    // Adjust dimax
    dimax = stdev::min(dimax - 1, svdnum - 1);
  }  // end if

  // Remove all small singular values
  svdval = svdval.subvec(0, dimax);
  svdu = svdu.cols(0, dimax);
  svdv = svdv.cols(0, dimax);

  // Calculate the reduced inverse from the SVD decompos
  return svdv*arma::diagmat(1/svdval)*svdu.t();
```

Portfolio Optimization Using *RcppArmadillo*

Fast portfolio optimization using matrix algebra can be implemented using *RcppArmadillo*.

```
// Fast portfolio optimization using matrix algebra and \emph{RcppArmadillo}
arma::vec calc_weights(const arma::mat& returns, // Asset returns
                       Rcpp::List controll) { // List of portfolio optimization parameters

  // Apply different calculation methods for weights
  switch(calc_method(method)) {
  case methodenum::maxsharpe: {
    // Mean returns of columns
    arma::vec colmeans = arma::trans(arma::mean(returns, 0));
    // Shrink colmeans to the mean of returns
    colmeans = ((1-alpha)*colmeans + alpha*arma::mean(colmeans));
    // Calculate weights using reduced inverse
    weights = calc_inv(covmat, dimax, eigen_thresh)*colmeans;
    break;
  }  // end maxsharpe
  case methodenum::maxsharpemed: {
    // Median returns of columns
    arma::vec colmeans = arma::trans(arma::median(returns, 0));
    // Shrink colmeans to the median of returns
    colmeans = ((1-alpha)*colmeans + alpha*arma::median(colmeans));
    // Calculate weights using reduced inverse
    weights = calc_inv(covmat, dimax, eigen_thresh)*colmeans;
    break;
  }  // end maxsharpemed
  case methodenum::minvarlin: {
    // Minimum variance weights under linear constraint
    // Multiply reduced inverse times unit vector
    weights = calc_inv(covmat, dimax, eigen_thresh)*arma::ones(ncols);
    break;
  }  // end minvarlin
  case methodenum::minvarquad: {
    // Minimum variance weights under quadratic constraint
    // Calculate highest order principal component
    arma::vec eigenval;
    arma::mat eigenvec;
```

Strategy Backtesting Using *RcppArmadillo*

Fast backtesting of strategies can be implemented using *RcppArmadillo*.

```
arma::mat roll_portf(const arma::mat& excess, // Asset excess returns
                     const arma::mat& returns, // Asset returns
                     Rcpp::List controll, // List of portfolio optimization model parameters
                     arma::uvec startp, // Start points
                     arma::uvec endd, // End points
                     double lambdaf = 0.0, // Decay factor for averaging the portfolio weights
                     double coeff = 1.0, // Multiplier of strategy returns
                     double bidask = 0.0) { // The bid-ask spread

  double lambda1 = 1-lambdaf;
  arma::uword nweights = returns.n_cols;
  arma::vec weights(nweights, fill::zeros);
  arma::vec weights_past = ones(nweights)/stdev::sqrt(nweights);
  arma::mat pnls = zeros(returns.n_rows, 1);

  // Perform loop over the end points
  for (arma::uword it = 1; it < endd.size(); it++) {
    // cout << "it: " << it << endl;
    // Calculate the portfolio weights
    weights = coeff*calc_weights(excess.rows(startp(it-1), endd(it-1)), controll);
    // Calculate the weights as the weighted sum with past weights
    weights = lambda1*weights + lambdaf*weights_past;
    // Calculate out-of-sample returns
    pnls.rows(endd(it-1)+1, endd(it)) = returns.rows(endd(it-1)+1, endd(it))*weights;
    // Add transaction costs
    pnls.row(endd(it-1)+1) -= bidask*sum(abs(weightv - weights_past))/2;
    // Copy the weights
    weights_past = weights;
  }  // end for

  // Return the strategy pnls
  return pnls;

}  // end roll_portf
```

Package *reticulate* for Running Python from RStudio

The package *reticulate* allows running Python functions and scripts from RStudio.

The package *reticulate* relies on Python for interpreting the Python code.

```
> # Install package reticulate
> install.packages("reticulate")
> # Start Python session
> reticulate::repl_python()
> # Exit Python session
> exit
```

You must set your Global Options in RStudio to your Python executable, for example: /Library/Frameworks/Python.framework/Versions/3.10/bin/python3.10

You can learn more about the package *reticulate* here:
https://rstudio.github.io/reticulate/

Running Python Under *reticulate*

```
"""
Script for loading OHLC data from a CSV file and plotting a candlestick plot.
"""
# Import packages
import pandas as pd
import numpy as np
import plotly.graph_objects as go
# Load OHLC data from csv file - the time index is formatted inside read_csv()
symbol = "SPY"
range = "day"
filename = "/Users/jerzy/Develop/data/" + symbol + "_" + range + ".csv"
ohlc = pd.read_csv(filename)
datev = ohlc.Date
# Calculate log stock prices
ohlc[["Open", "High", "Low", "Close"]] = np.log(ohlc[["Open", "High", "Low", "Close"]])
# Calculate moving average
lookback = 55
closep = ohlc.Close
pricema = closep.ewm(span=lookback, adjust=False).mean()
# Plotly simple candlestick with moving average
# Create empty graph object
plotfig = go.Figure()
# Add trace for candlesticks
plotfig = plotfig.add_trace(go.Candlestick(x=datev,
  open=ohlc.Open, high=ohlc.High, low=ohlc.Low, close=ohlc.Close,
  name=symbol+" Log OHLC Prices", showlegend=False))
# Add trace for moving average
plotfig = plotfig.add_trace(go.Scatter(x=datev, y=pricema,
  name="Moving Average", line=dict(color="blue")))
# Customize plot
plotfig = plotfig.update_layout(title=symbol + " Log OHLC Prices",
  title_font_size=24, title_font_color="blue", yaxis_title="Price",
  font_color="black", font_size=18, xaxis_rangeslider_visible=False)
# Customize legend
plotfig = plotfig.update_layout(legend=dict(x=0.2, y=0.9, traceorder="normal",
  itemsizing="constant", font=dict(family="sans-serif", size=18, color="blue")))
# Render the plot
plotfig.show()
```

Homework Assignment

Required

Study all the lecture slides in `FRE7241_Lecture_6.pdf`, and run all the code in `FRE7241_Lecture_6.R`

Recommended

- Read about *estimator shrinkage*:
  Aswani Regression Shrinkage Bias Variance Tradeoff.pdf
  Blei Regression Lasso Shrinkage Bias Variance Tradeoff.pdf
- Read about *optimization methods*:
  Bolker Optimization Methods.pdf
  Yollin Optimization.pdf
  DEoptim Introduction.pdf
  Ardia DEoptim Portfolio Optimization.pdf
  Boudt DEoptim Portfolio Optimization.pdf
  Boudt DEoptim Large Portfolio Optimization.pdf
  Mullen Package DEoptim.pdf
- Read about *momentum*:
  Bouchaud Momentum Mean Reversion Equity Returns.pdf